DYNAMIC EVENT TICKETING MARKETPLACE WITH PREDICTIVE SCALPING MITIGATION ALGORITHMS

PROJECT REPORT

Submitted by

DEEPAK A (7376242AD136)

THIRUMURUGAN K (7376242AL210)

In partial fulfilment for the award of the degree

of

BACHELOR OF TECHNOLOGY

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING



BANNARI AMMAN INSTITUTE OF TECHNOLOGY

(An Autonomous Institution Affiliated to Anna University, Chennai)

SATHYAMANGALAM-638401

ANNA UNIVERSITY: CHENNAI 600 025

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this project report "**DYNAMIC EVENT TICKETING MARKETPLACE WITH PREDICTIVE SCALPING MITIGATION ALGORITHMS**" is the bonafide work of "**DEEPAK A (7376242AD136),THIRUMURUGAN K (7376242AL210)**" who carried out the project work under my supervision.

SIGNATURE

Dr. A. Bharathi

HEAD OF THE DEPARTMENT

Department of Artificial Intelligence and Machine Learning

Bannari Amman Institute of Technology

Sathyamangalam-638401

SIGNATURE

Prof. Balamurugan . E

ASSISTANT PROFESSOR

Department of Artificial Intelligence and Machine Learning

Bannari Amman Institute of Technology

Sathyamangalam-638401

Submitted for Project Viva Voce examination held on ……

Internal Examiner I **Internal Examiner II**

DECLARATION

We affirm that the project work titled **"Dynamic Event Ticketing Marketplace With Predictive Scalping Mitigation Algorithms"** being submitted in partial fulfilment for the award of the degree of **Bachelor of Technology** in **Artificial Intelligence and Machine Learning** is the record of original work done by us under the guidance of **Prof. Balamurugan E**, Assistant Professor level I I , Department of Artificial Intelligence and Machine Learning. It has not formed a part of any other project work(s) submitted for the award of any degree or diploma, either in this or any other University.

DEEPAK A	**THIRUMURUGAN K**
(7376242AD136)	**(7376242AL210)**

I certify that the declaration made above by the candidates is true.

(Signature of the Guide)

Prof. Balamurugan E

ACKNOWLEDGEMENT

We express our sincere gratitude to our esteemed Chairman **Dr. S. V. Balasubramaniam** and the respected Principal **Dr. C. Palanisamy** for providing the academic environment, facilities and support required to carry out this project.

We are grateful to **Dr. Bharathi A**, **Head of the Department, Department of Artificial Intelligence and Machine learning**, for his valuable suggestions to carry out the project work successfully.

We wish to express our sincere thanks to Faculty guide **Prof. Balamurugan E**, **Assistant Professor level II , Department of Artificial Intelligence and Machine learning** , for his constructive ideas, inspirations, encouragement, excellent guidance, and much needed technical support extended to complete our project work.

We would like to thank our friends, faculty and non-teaching staff who have directly and indirectly contributed to the success of this project.

DEEPAK A (7376242AD136)
THIRUMURUGAN K (7376242AL210)

ABSTRACT

Choosing a suitable technology career and identifying the skills required to reach it posed a challenge for students with uneven exposure to programming languages, tools and applied concepts. Existing career guidance and learning platforms often addressed role selection and content delivery separately. This project developed CareerCompass AI as a prototype connecting technical skill-profile analysis with a structured learning workspace. The career-analysis component accepted technical-skill text, normalized the input, converted it into Term Frequency-Inverse Document Frequency features and applied a multi-class Logistic Regression classifier to rank likely roles. It returned up to three candidate roles, confidence values, matched and missing skill tokens, and a token-based coverage percentage. The curated dataset contained 440 labelled skill profiles across 11 technology roles, with 40 profiles per role. The repository-reported five-fold stratified cross-validation benchmark showed 78.86% mean accuracy with a standard deviation of 1.54%; macro precision, recall and F1-score were 0.8014, 0.7886 and 0.7833. The learning component requested a six-to-eight-node prerequisite graph from an external generative service and supported concept explanations, practice tasks, learning resources, prerequisite-based unlocking, completion tracking, recommendations and progress visualizations. The resource workflow searched educational videos, retrieved available transcripts and selected a short segment; submitted practice answers could receive automated scores and feedback. The results demonstrated the feasibility of connecting role hypotheses, vocabulary-based gap summaries and AI-assisted learning functions. They did not establish live labour-market demand or employment outcomes. The skill-role dataset remained curated and static, the coverage measure represented vocabulary overlap rather than proficiency, and a stricter evaluation was needed because vectorization preceded the documented cross-validation folds. The prototype was therefore positioned as educational decision support. Live job-posting aggregation, integrated end-to-end testing and user-centred evaluation remained future work.

Keywords: Career recommendation; skill gap analysis; text classification; knowledge graph; personalized learning path.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT	v
	ABSTRACT	vi
	TABLE OF CONTENTS	vii
	LIST OF FIGURES	x
	LIST OF TABLES	xi
	ABBREVIATIONS AND NOMENCLATURE	xii
1	**INTRODUCTION**	**1**
1.1	Background of the Work	1
1.2	Problem Statement	2
1.3	Motivation	3
1.4	Objectives of the Proposed Work	3
1.5	Scope of the Proposed Work	4
1.6	Contributions of the Work	5
	SUMMARY	**5**
2	**LITERATURE SURVEY**	**6**
2.1	Skill Gaps and Changing Career Requirements	6
2.2	AI-Based Career Recommendation	7
2.3	Text Representation and Classification	7
2.4	Knowledge Graphs and Learning Path Recommendation	8
2.5	Generative AI and Learning Resource Support	9
2.6	Summary of Relevant Literature	10
	Identified Research Gap and Problem Definition	10

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
3	**OBJECTIVES AND METHODOLOGY**	**11**
3.1	Objectives	11
3.2	Requirements Analysis	11
3.3	System Architecture and Workflow	12
3.4	Career Prediction Methodology	14
3.5	Learning Graph Methodology	16
3.6	Tools, Technologies and Deployment	17
3.7	Evaluation Methodology	18
3.8	Detailed Career Prediction Workflow	19
3.9	Dataset Structure and Class Coverage	20
3.10	Text Feature Construction	21
3.11	Model Selection and Evaluation Protocol	21
3.12	Skill-Gap Calculation Procedure	22
3.13	Learning-Graph Generation and Validation	23
3.14	Learning Resources, Video Snippets and Practice	24
3.15	API Boundaries, Persistence and Security	24
3.16	Reproducibility and Evidence Limitations	25
3.17	Test Design and Error Handling	25
4	**PROPOSED WORK MODULES**	**27**
4.1	Career Profile and Role Prediction Module	27
4.2	Skill Gap and Coverage Analysis Module	28
4.3	Personalized Knowledge Graph Module	29

TABLE OF CONTENTS (CONTINUED)

CHAPTER NO.	TITLE	PAGE NO.
4.4	Learning Progress and Recommendation Module	30
4.5	Resource, Video Snippet and Practice Modules	32
4.6	User Management and Data Persistence	33
4.7	API Design and Data Model	34
5	**RESULTS AND DISCUSSION**	**36**
5.1	Dataset and Model Results	36
5.2	Automated Test Results	38
5.3	Functional Review of Learning Modules	39
5.4	Strengths and Limitations	40
5.5	Cost-Benefit Analysis	41
6	**CONCLUSIONS AND SUGGESTIONS FOR FUTURE WORK**	**42**
6.1	Conclusion	42
6.2	Suggestions for Future Work	42
	REFERENCES	**44**
	APPENDICES	**46**

LIST OF FIGURES

SI. NO.	FIGURE NAME	PAGE NO.
1.1	Main dashboard and recommendations	2
3.1	Conceptual modular architecture	13
3.2	End-to-end workflow	14
3.3	Model inference and skill-coverage pipeline	16
4.1	Career skill-input interface	28
4.2	Predicted roles and skill-gap results	28
4.3	Career advisor and role knowledge base	29
4.4	Interactive learning knowledge graph	30
4.5	Learning dashboard and recommendations	31
4.6	Skill Assessment: selected video-editing topic	32
4.7	Resource explorer and video snippet	33
4.8	Practice feedback and correctness explanation	33
5.1	Mean classifier accuracy under five-fold cross-validation	37

LIST OF TABLES

NO.	TITLE	PAGE NO.
2.1	Summary of relevant literature	10
2.2	Research gaps and project response	10
3.1	Functional requirements	11
3.2	Non-functional requirements	12
3.3	Role-label sample distribution	14
3.4	Technology stack and role in the project	17
3.A	Illustrative role-skill inference pipeline stages	19
3.B	Role-label distribution in the supplied dataset	20
3.C	Learning-graph generation constraints recorded in the supplied code	23
4.1	Primary API endpoints and response summary	34
4.2	Learning data entities and their purposes	35
5.1	Dataset characteristics	36
5.2	Classifier benchmark results under five-fold stratified cross-validation	37
5.3	Automated career API test results	38
5.4	End-to-end verification matrix for the learning modules	39
5.5	Strengths, limitations and recommended responses	40
5.6	Qualitative cost-benefit analysis	41

ABBREVIATIONS AND NOMENCLATURE

ABBREVIATION	EXPANSION
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
CV	Cross-Validation
DF	Document Frequency
IDF	Inverse Document Frequency
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
NLP	Natural Language Processing
OAuth	Open Authorization
ORM	Object-Relational Mapping
REST	Representational State Transfer
SPA	Single-Page Application
SQL	Structured Query Language
TF	Term Frequency
TF-IDF	Term Frequency-Inverse Document Frequency
UI	User Interface
URL	Uniform Resource Locator
WEF	World Economic Forum

CHAPTER - I

INTRODUCTION

1.1 BACKGROUND OF THE WORK

The rapid digitisation of education and work has increased the amount of technical learning material available to students. At the same time, career roles are described through evolving combinations of languages, frameworks, data platforms, engineering practices and domain knowledge. The World Economic Forum's Future of Jobs Report 2025 identifies skill gaps as a major barrier to workforce transformation and highlights the need for reskilling and upskilling as technology changes job tasks [1]. For a student, this translates into two connected questions: which career directions best match the skills already acquired, and what should be learned next to become better prepared for a selected direction.

Career decisions are often made using broad interest surveys, informal advice, job titles or isolated course lists. These approaches may give a useful starting point, but they do not always show a transparent mapping between a student's skills and a target role's requirements. A student who knows Python, SQL and basic statistics may be suited to explore several data-oriented roles, yet the next learning steps for a data analyst differ from those for a data engineer or machine-learning engineer. A practical system should make alternatives visible, describe the evidence behind the recommendation and identify the competencies that appear absent from the supplied profile.

CareerCompass AI is designed to connect role exploration with skill development. The career-analysis module interprets a short technical-skill profile and predicts likely job-role categories. It also compares the supplied skill tokens against a role-specific vocabulary and reports matched skills, missing skills and a coverage percentage. The learning workspace then provides a structured path for a learner-selected topic: a prerequisite knowledge graph, concept explanations, practice tasks, resources, feedback and progress tracking. These capabilities are intended to help students move from a high-level career suggestion to an actionable self-learning plan. The dashboard linking learning progress and recommendations is shown in Figure 1.1.

The project uses a supervised text-classification model for the role-prediction task and a generative- AI service for the creation of learning content. These methods serve different purposes. The classifier is trained on labeled skill-profile examples and returns probability scores across predefined classes. The generative service builds a topic-specific graph and supporting practice content. Because the learning graph is generated from an external model, its output must be checked for logical ordering, factual accuracy and appropriate resource choices rather than treated as an infallible curriculum.

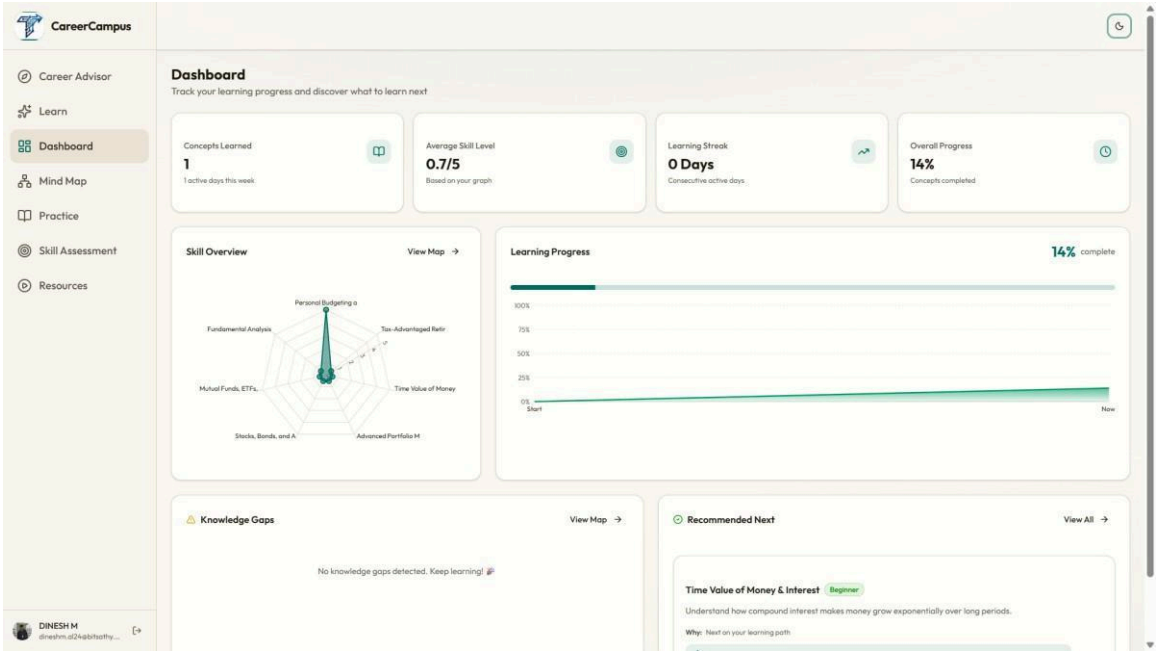


Figure 1.1. CareerCompass AI main dashboard showing learning progress and recommendations.

1.2 PROBLEM STATEMENT

Students frequently lack an evidence-based and actionable route from their current technical skill set to a plausible career direction. A role recommendation alone does not reveal which competencies are already represented, which are missing according to a target-role skill vocabulary, or how those competencies could be learned in a manageable sequence. Conversely, a generic learning platform may provide abundant resources without connecting them to a specific career goal or prerequisite chain.

The problem addressed by this project is the design of a student-oriented decision-support platform that links skill-profile-based career recommendation, explicit skill-gap reporting and a personalized, progress-aware learning path. The project title also refers

to skill-demand aggregation. In the current implementation, role requirements are represented by a curated, static skill-profile dataset and a vocabulary derived from those records; automated collection and aggregation of live job-posting demand have not been verified in the source code and are therefore treated as planned work, not as a completed capability.

1.3 MOTIVATION

Three observations motivate the proposed work. First, students may have difficulty comparing multiple technical roles because role descriptions use overlapping terminology and different levels of specificity. Second, a simple percentage or list of missing items is more actionable when paired with an ordered learning path, practice exercises and feedback. Third, learning progress should be visible over time so that the student can decide whether to continue a topic, revisit a weak concept or move to the next prerequisite. A platform that connects these elements can reduce the manual effort required to assemble a plan from separate websites and tools.

The career classifier also provides an opportunity to make the decision process measurable. The repository includes an evaluation comparing Logistic Regression, Random Forest, Gradient Boosting, XGBoost and LightGBM under five-fold stratified cross-validation. The selected Logistic Regression model

achieved the strongest reported mean accuracy among those candidates. The learning workspace complements this statistical output with graph-based representation of concepts and state tracking. Together, the components support a more complete guidance flow while retaining the need for human judgement and careful evaluation.

1.4 OBJECTIVES OF THE PROPOSED WORK

1. Develop a text-based interface for capturing a learner’s current technical skills in a simple, validated format.
2. Predict likely technology career roles from the entered skill profile using a supervised multi-class classifier.

3. Return the highest-ranked role and up to two alternative roles, along with model confidence and an uncertainty indication when the top probability is below the configured threshold.
4. Compare the entered skills against the role-skill knowledge base and report matched skills, missing skills, total required vocabulary and token-based coverage percentage.
5. Generate a topic-specific learning knowledge graph containing foundational and advanced concepts linked by prerequisite edges.
6. Provide supporting resources, AI-generated practice questions, answer grading and explanatory feedback to support independent learning.
7. Track concept completion, practice completion, daily learning activity, progress percentages and recommended next topics.
8. Document evaluation evidence, implementation limitations, data-quality risks and a practical pathway for future live demand aggregation and end-to-end integration.

1.5 SCOPE OF THE PROPOSED WORK

The current scope covers technical-role prediction across 11 supported job categories; skill matching and missing-token analysis; a browsable role knowledge base; learning-goal creation and switching; AIgenerated knowledge graphs; prerequisite-based concept status; practice problems and AI grading; learningresource display; transcript-guided video snippet selection; and dashboard/recommendation functions. The career-analysis dataset contains 440 records, each represented by a skill string and a job-role label. The role predictor is a supervised text classifier, not a general labour-market forecasting model.

The learning path component is built around a web interface and a FastAPI service. Its persistence layer uses SQLAlchemy models with PostgreSQL as the configured database and SQLite as a localdevelopment fallback. The code includes password authentication and Google sign-in support. The learning graph generator calls the Gemini API, and the video-resource workflow uses YouTube search and transcript

tooling. Configuration and deployment files are present, but availability of external APIs depends on credentials, network access, quota and service behaviour.

The following items are outside the validated current scope: a continuously refreshed job-vacancy corpus; reliable employer-specific or geographical demand indicators; salary prediction; candidate

screening; job-placement guarantees; formal aptitude/personality assessment; evidence-based proficiency measurement; and a fully verified, single sign-on journey across every module. These may be considered in future development after data collection, integration design and evaluation.

1.6 CONTRIBUTIONS OF THE WORK

A practical supervised baseline for mapping short skill profiles to one of 11 technical career categories, with explicit cross-validation metrics and alternate predictions.

A transparent, role-specific skill-gap view that separates matched vocabulary from missing vocabulary and calculates a simple coverage ratio.

A prerequisite-linked learning graph with concept status, explanations, resources and practice material generated around a learner-specified topic.

Progress-oriented tools including completion tracking, activity streaks, dashboard summaries and prioritized next-step recommendations.

A documented evaluation and limitation analysis that distinguishes model benchmark metrics from unmeasured user outcomes and from future labour-market data ingestion.

SUMMARY

CareerCompass AI addresses the challenge of connecting a student's current skill profile to potential roles and actionable learning steps. The prototype combines role classification, vocabulary-level gap analysis and an AI-assisted learning path; live job-demand aggregation remains future work.

CHAPTER - II

LITERATURE SURVEY

This chapter reviews literature and technical documentation relevant to the main functions of CareerCompass AI: changing skill requirements, AI-based job recommendation, text representation and classification, learning-path recommendation through knowledge graphs, and generative-AI support for educational resources. The review is intended to establish design context and identify the gap addressed by the current prototype. External research findings are not used as performance claims for this project; the quantitative results reported in Chapter 5 come from the supplied dataset and implementation artefacts.

2.1 SKILL GAPS AND CHANGING CAREER REQUIREMENTS

Career preparation increasingly involves continuous learning rather than one-time qualification. The World Economic Forum's Future of Jobs Report 2025 draws on employer perspectives across industries and economies and reports that many employers expect skill requirements to change during the 2025-2030 period. It identifies skills gaps as a major obstacle to workforce transformation and highlights both technology-related and human skills [1]. This context supports the value of tools that help learners inspect their current skill inventory and create a focused plan for development.

However, a report about labour-market trends is not equivalent to a live vacancy dataset. Published surveys summarize aggregated employer views at a point in time; individual job advertisements may have distinct regional, organizational and seniority requirements. A career-guidance prototype that uses a static labeled dataset can demonstrate the mechanics of role classification, but it cannot safely claim current market demand unless the underlying job-posting sources, collection dates, extraction rules and update process are documented. This distinction informs the scope and limitations of the present project.

2.2 AI-BASED CAREER RECOMMENDATION

AI-based job recommendation systems attempt to rank roles or vacancies based on representations of job seekers, their skills, experience and preferences. A 2023 survey published at the International Conference on Sustainable Computing and Data Communication Systems discusses machine-learning and deep-learning approaches and identifies challenges such as sparse data, privacy, bias, transparency and interpretability [2]. These issues matter for student-facing guidance because a probability score may look authoritative even when the training data are limited or do not reflect a real hiring population.

A 2023 study titled JobEdKG explores knowledge-graph-based recommendations of online courses and the prediction of skills associated with career choices, illustrating the relationship between a target career and the learning resources needed to develop its skills [3]. This is relevant to CareerCompass AI because role recommendation is more useful when it is connected to a learning plan. The present prototype uses a simpler approach: a TF-IDF classifier ranks role labels and a rule-based skill vocabulary reports potential gaps; the learning component then generates a separate prerequisite graph for a selected topic. It does not reproduce JobEdKG's knowledge-graph algorithms or claim comparable results.

The design therefore emphasizes a transparent baseline rather than an opaque end-to-end recommendation model. Its top-three output can expose alternative role possibilities; the confidence

threshold can flag uncertain predictions; and the skill-gap output can show why a target role may require further study. Such outputs should support reflection and discussion, not automatically eliminate career choices or make eligibility decisions.

2.3 TEXT REPRESENTATION AND CLASSIFICATION

Skill profiles are supplied as short text strings containing words such as Python, SQL, React, Kubernetes, testing or TensorFlow. TF-IDF is a standard information-retrieval representation that assigns weights to terms according to their frequency in a document and their distribution across a collection. Salton and Buckley describe term-weighting approaches and their role in representing text for retrieval [4]. The scikit-learn

TfidfVectorizer implements this family of transformation and can produce sparse feature matrices suitable for machine-learning algorithms [5].

Logistic Regression is a discriminative classifier that can produce class probabilities when configured for probabilistic prediction. In the project, the vectorized skill string is passed to a multi-class Logistic Regression model, and the probability values are sorted to obtain the top three role candidates. Scikit-learn documents the LogisticRegression estimator and its multiclass behaviour [6]. A practical advantage of this model is that it is comparatively efficient and uses established tooling; a limitation is that its learned decision boundary depends on the coverage and quality of the vocabulary in the training data.

The supplied dataset has 440 profiles across 11 balanced classes. Because all examples are skill strings paired with role labels, measured performance indicates how well the model distinguishes those categories within the constructed dataset. It does not establish that the predicted role is the learner's best career choice or that the listed missing tokens are the only competencies required by employers. The role-coverage calculation further simplifies skill matching into normalized tokens and should be interpreted as a heuristic rather than a formal assessment of proficiency.

2.4 KNOWLEDGE GRAPHS AND LEARNING PATH RECOMMENDATION

A learning knowledge graph represents learning concepts as nodes and relationships such as prerequisites as directed edges. It can communicate which concepts should be studied first and which topics become available after prerequisites have been completed. Liu et al. describe a learning-path recommendation system based on a knowledge graph and discuss the value of structuring concepts and their predecessor-successor relationships [7]. Related research on personalized learning-path recommendations has examined combinations of knowledge-graph representations, interaction patterns and resource recommendations [8]. A 2026 survey further reviews knowledge-graph-based personalized learning paths and highlights explainability and the alignment between learner characteristics and path generation as continuing research issues [9].

CareerCompass AI adopts the graph representation as a practical interface and progress model. For a topic supplied by the learner, the generative service is prompted to return six to eight concepts, prerequisite edges, short explanations, practice tasks and resource metadata. The graph interface supports zooming, panning, node selection and status visualization. The backend stores nodes and edges and checks prerequisite completion when deciding whether a concept is locked or recommended. The approach improves readability

of a learning sequence, but generated prerequisites still require review because a language model can omit foundational concepts, introduce invalid relationships or generate content of uneven quality.

2.5 GENERATIVE AI AND LEARNING RESOURCE SUPPORT

Generative AI can create explanations and practice material in response to a learner's chosen topic. The Gemini API supports content generation and structured output modes that help applications request machine-readable responses [10]. In the current system, the graph generator requests JSON containing nodes, edges, concepts, problem statements and resource titles. An answer-grading endpoint can return a score, feedback and a model answer for a submitted response. The value of this approach is the ability to create topic-specific material without manually authoring every possible path; the risk is that plausiblesounding output may be incorrect, inconsistent or insufficiently grounded in a verified curriculum.

The YouTube snippet service adds a resource-discovery step. It searches for candidate videos, attempts to retrieve transcripts and asks the generative model to identify a relevant continuous segment of roughly 60 to 90 seconds. If no transcript is available, the code provides a fallback response. This design aims to reduce the time needed to locate a useful explanation, but it depends on video availability, transcript quality, external-service behaviour and the language model's timestamp selection. It should be treated as a navigation aid, not as evidence that the video is authoritative or pedagogically optimal. Table 2.1 summarizes the literature reviewed in this chapter and its relevance to CareerCompass AI.

2.6 SUMMARY OF RELEVANT LITERATURE

Table 2.1. Summary of relevant literature.

Source	Main focus	Relevance to CareerCompass AI	Difference / limitation
[1] World Economic Forum, 2025	Employer views on job and skill changes	Establishes the importance of upskilling and changing skill requirements	A report is not the project's own live demand dataset.
[2] AI job- recommendation survey, 2023	Methods and challenges in AI job recommendations	Motivates transparent output and attention to bias, privacy and interpretability	Survey findings are not direct validation of this prototype.
[3] JobEdKG, 2023	Career-linked skill and course recommendation using knowledge graphs	Supports linking roles, skills and learning resources	The current prototype uses a simpler classifier and graph generator.
[7] Liu et al., 2023	Knowledge-graph learning-path design	Supports prerequisite-linked concepts and learning sequence visualization	The current graph is generated by a language model, not the paper's optimization algorithm.
[8] IEEE BigData, 2024	Personalized learning paths with knowledge graph and sequence methods	Shows research interest in individualized resource sequences	The prototype does not implement that hybrid algorithm.
[9] Li et al., 2026	Survey of personalized learning-path recommendations	Highlights explainability, evaluation and personalization issues	The current system has limited learner-history evidence.

Source	Main focus	Relevance to CareerCompass AI	Difference / limitation
[4]-[6], technical references	TF-IDF and Logistic Regression	Grounds the selected text representation and classifier	Performance depends on training-data construction and validation hygiene.

IDENTIFIED RESEARCH GAP AND PROBLEM DEFINITION

The review suggests a practical gap between role-level recommendations and the detailed learning actions needed after a recommendation. Career-recommendation systems may rank roles without showing a structured path to close the gap; learning platforms may offer paths without starting from a skill-profilebased view of career alternatives. A learner-centred tool should connect these steps, show the supporting information and make progress visible. It must also clearly separate curated skill requirements from verified live labour-market demand. Table 2.2 maps these gaps to the prototype response and remaining issues.

Table 2.2. Research gaps and project response.

Identified gap	Project response	Remaining issue
Career suggestions do not always show alternative role hypotheses.	Return top-three candidates with confidence values.	Predictions are limited to 11 labels and the current dataset.
A role label does not explain skill readiness.	Return matched tokens, missing tokens and a coverage percentage.	Token matching is not a competency or proficiency assessment.
Generic resources can feel unstructured.	Generate concept nodes with prerequisite edges and learning resources.	Graph quality and prerequisite correctness require human review.
Learners need feedback and visible progress.	Provide practice, AI grading, completion history and dashboard indicators.	Learning gain and usability have not been experimentally measured.
Skill-demand claims require fresh market evidence.	Current role skills are represented by the curated labeled dataset.	Live job-posting ingestion and demand aggregation remain future work.

CHAPTER - III

OBJECTIVES AND METHODOLOGY

3.1 OBJECTIVES

The methodology maps the objectives into a sequence of user input, data processing, model inference, gap calculation, learning content generation and progress feedback. Two types of intelligence are used: a trained supervised model produces career-role scores, while a generative language model creates topicspecific learning content. The two are complementary and should not be confused: the former is benchmarked with labeled examples; the latter is assessed through content quality and functional verification, for which a controlled quantitative evaluation is still required. The functional and non-functional requirements are summarized in Tables 3.1 and 3.2.

3.2 REQUIREMENTS ANALYSIS

3.2.1 Functional requirements

Table 3.1. Functional requirements.

ID	Requirement	Expected behaviour
FR-01	Skill entry	Accept a non-empty text list of technical skills; reject whitespace-only or non-letter input.
FR-02	Role classification	Return a top predicted role and up to three ranked role candidates with confidence values.
FR-03	Skill gap report	Return matched and missing skills, total role vocabulary size and a coverage percentage.
FR-04	Role exploration	List supported job roles and their prerequisite skill vocabularies.
FR-05	Learning goal creation	Accept a topic and request a structured, topic-specific knowledge graph.
FR-06	Concept exploration	Show concept description, explanation, related concepts, resources and practice problems.
FR-07	Prerequisite control	Lock concepts when required prerequisite nodes are incomplete; recommend eligible nodes.
FR-08	Learning tracking	Store concept completion, practice completion and daily activity for progress views.
FR-09	Recommendations	Rank next topics and identify quick-win concepts based on path state.
FR-10	Practice support	Generate or display practice items and return AI-supported score/feedback when submitted.
FR-11	Resource support	List resources and optionally find a transcript-grounded educational-video segment.
FR-12	User access	Support email/password accounts and Google sign-in in the learning service.

3.2.2 Non-functional requirements

Table 3.2. Non-functional requirements.

Requirement	Description and importance
Usability	Skill entry, role comparisons and learning concepts should be understandable to students with varied experience.
Responsiveness	The web interfaces should remain usable on standard desktop and mobile viewport widths; practical cross-device testing is still required.
Reliability	Missing API keys, unavailable transcript sources or failed model calls should produce clear errors or documented fallbacks.
Performance	The career classifier should load model artefacts once per API process and use an in-memory prediction path for requests.
Security	API keys remain in environment variables; password hashes and signed tokens are used by the learning service; production CORS and access policies require hardening.
Maintainability	The services expose documented JSON interfaces, use Pydantic schemas and keep core model logic separate from HTTP handlers.
Traceability	Model performance, model version and source of role-skill information should be documented so recommendations can be interpreted.
Scalability	Current scale is a prototype; production traffic, model-serving concurrency, API quota limits and database load require deployment testing.

3.3 SYSTEM ARCHITECTURE AND WORKFLOW

The architecture is organized into user-interface modules, API services, machine-learning artefacts, persistent learning data and external content/AI services. The career analysis interface collects skills and requests predictions from the career-intelligence API. The learning workspace creates goals, retrieves graph nodes and edges, records progress and displays resources. The career prediction service reads the trained TF- IDF vectorizer, Logistic Regression classifier, label encoder and skill-profile dataset. The learning service uses FastAPI with SQLAlchemy-backed data models and the configured relational database. The Gemini API is called for graph generation and answer grading; the YouTube service locates candidate videos and transcript segments. Figure 3.1 summarizes these components and their relationships.

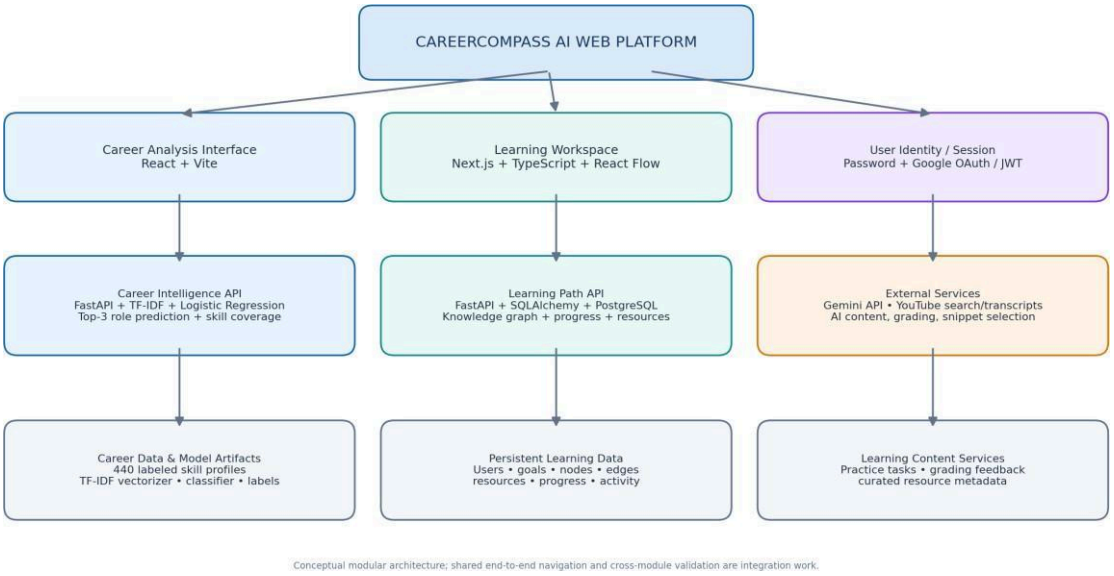


Figure 3.1. Conceptual modular architecture of CareerCompass AI.

The architecture represents the intended career-to-learning journey rather than a verified singleprocess deployment. The code contains the core services and interface modules, but shared navigation, single identity across every interface, and cross-module end-to-end behaviour must be tested together before the system can be treated as a unified production application. This qualification does not alter the module-level features described below; it defines the level of integration evidence currently available. Figure

3.2 traces the learner workflow from skill input to learning progress and marks live demand aggregation as a proposed extension.

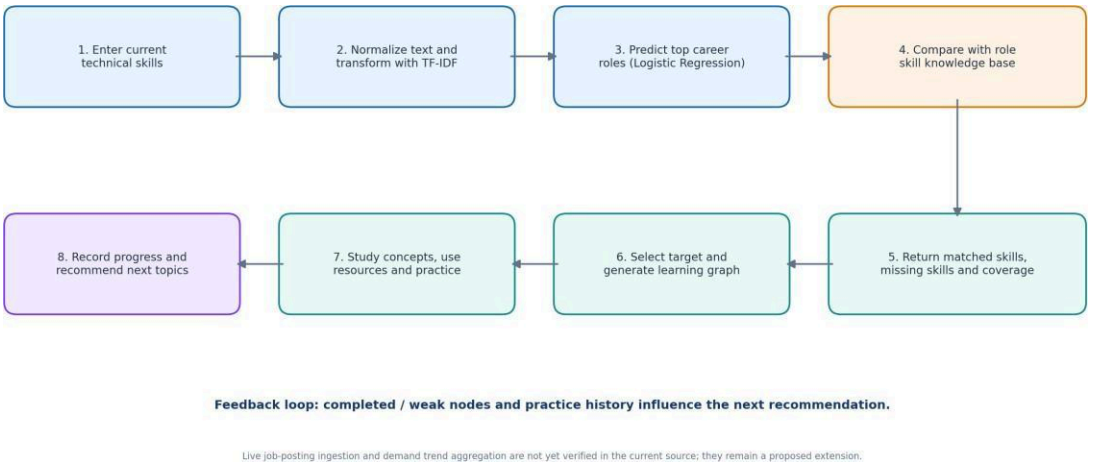


Figure 3.2. End-to-end workflow from skill profile to learning progress. Live demand aggregation is a proposed extension.

3.4 CAREER PREDICTION METHODOLOGY

3.4.1 Dataset construction

The supplied CSV contains 440 rows and two columns: `skills` and `job_role`. Each of 11 labels has 40 examples. The rows are short skill-profile strings. There are no missing values in the inspected dataset.

The data artefacts do not identify the examples as a representative sample of live vacancy data; therefore, the dataset is described as a curated skill-to-role dataset. The class balance is useful for benchmarking a baseline classifier, but performance on future users depends on whether the user vocabulary resembles the training examples. Table

3.3 shows the number of supplied records for each role label.

Table 3.3. Role-label sample distribution.

Role label	Records	Role label	Records
AI Engineer	40	Backend Developer	40
Data Analyst	40	Data Engineer	40
Data Scientist	40	Database Administrator	40
DevOps Engineer	40	Frontend Developer	40
ML Engineer	40	Mobile Developer	40
QA Engineer	40	Total	440

3.4.2 Text preprocessing and vectorization

The preprocessing function converts input to lowercase, replaces non-alphanumeric characters with spaces and normalizes repeated whitespace. The saved TF-IDF vectorizer uses word-level features, oneword and two-word n-grams, a maximum of 250 features, smoothed inverse document frequency and L2 normalization. The saved artefact inspected for this report has `sublinear_tf=False`; the repository's descriptive feature summary mentions `sublinear_tf=True`, so the training documentation should be reconciled with the saved artefact before a future retraining release. The report uses the parameters recorded in the actual model artefact for describing inference.

For a term t and profile d, TF-IDF combines a term-frequency value with an inverse-documentfrequency value so that common terms can be down-weighted relative to more informative terms. In a vectorized representation, each profile becomes a sparse numerical feature vector. The selected vectorizer uses up to 250 dimensions, so skills that do not map to its learned vocabulary do not contribute meaningful evidence to the classifier.

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t),\ IDF(t) = \log((1 + N) / (1 + DF(t))) + 1$$

3.4.3 Classifier and top-three recommendation

The target labels are converted to numeric class indices with a LabelEncoder. The selected classifier is Logistic Regression with `max_iter=500`, `class_weight="balanced"` and `random_state=42`. The classifier estimates a probability for each class. The API sorts these probabilities in descending order and returns the top three candidate roles. If the first candidate probability is below 0.40, the response sets an uncertainty flag and still returns alternatives, allowing the interface to avoid presenting a weakly supported single prediction as certain.

The model probabilities should be interpreted as the classifier's relative scores for the labels represented in the training data. They are not a calibrated estimate of employment success or a measure of a The inference stages and vocabulary-coverage calculation are summarized in Figure 3.3.

student's future performance. The confidence threshold is a software rule for identifying uncertain outputs, not an independently validated decision threshold.



Figure 3.3. Model inference pipeline and role-skill coverage calculation.

3.4.4 Skill-gap calculation

The career predictor builds a skill knowledge base by collecting all unique space-separated tokens found in the training examples for each role. At inference time it normalizes the entered text to lowercase and splits it on spaces and commas. Matched skills are the intersection between the supplied token set and the target-role set; missing skills are target-role tokens that do not appear in the user input. Coverage is calculated using the ratio below. The method is simple and interpretable, but token-level splitting can divide multiword competencies into several terms and does not check proficiency, recency, certification or practical evidence.

Coverage (%) = (Number of matched role-skill tokens / Total role-skill tokens) × 100

The coverage score is only meaningful relative to the vocabulary derived from the current dataset. For example, it should be described as “coverage of the role-skill vocabulary” rather than “percentage job-ready.” A future version should use a curated taxonomy of distinct skills, synonyms, versions and competency levels, and should validate role requirements against multiple dated and traceable labour-market sources.

3.5 LEARNING GRAPH METHODOLOGY

When a learner submits a topic, the learning API asks the Gemini service to create a JSON graph containing six to eight nodes. Each node includes a stable identifier, label,

description, beginner explanation, initial status, level, practice problems, related topics and resource metadata. The prompt requires each nonroot node to have at least one prerequisite edge, with the first one or two nodes initially recommended and later nodes locked. The graph-generation code limits the number of retained resources by type so that a node does not accumulate an unbounded list.

The backend converts graph identifiers into stored node identifiers, places nodes on the canvas and stores directed prerequisite edges. The mind-map API retrieves nodes and edges and derives display states from completion records. A node with uncompleted prerequisites is locked; a node whose prerequisites are satisfied can be made recommended. Completing a node stores a user-progress record and records daily activity for streak calculation. Users can create several learning goals, switch between them and remove a goal with its associated graph.

Resource and practice services add learning support to the graph. The YouTube service searches for candidate videos, retrieves a transcript when available and requests a short relevant segment from the language model. The grading endpoint requests an evaluation of a learner's answer and returns feedback and a model answer. These functions are externally dependent and should include an explicit human-review path for content that is incorrect, ambiguous or not appropriate to the learner's level. Table 3.4 maps the implementation technologies to their project roles.

3.6 TOOLS, TECHNOLOGIES AND DEPLOYMENT

Table 3.4. Technology stack and role in the project.

Layer / purpose	Technology in implementation	Role in the project
Career-analysis UI	React, Vite, Tailwind CSS	Skill input, result cards, coverage indicators and role knowledge base.
Learning workspace UI	Next.js [12], React, TypeScript, Tailwind CSS	Pages for goals, graph, dashboard, resources, practice and recommendations.
Graph visualization	@xyflow/react (React Flow) [13]	Interactive nodes and directed prerequisite edges.
Charts and progress UI	Recharts and SVG/CSS elements	Dashboard charts, progress summaries and visual feedback.
API services	Python, FastAPI [11], Pydantic	JSON interfaces and request/response validation.
Career model	scikit-learn, joblib, pandas, NumPy	Text preprocessing support, TF-IDF transformation, role classification and artefact loading.
Learning persistence	SQLAlchemy, PostgreSQL 15; SQLite local fallback	Stores user data, goals, nodes, edges, resources and progress.
Generative AI	Gemini API	Learning graph creation, practice content and answer grading.

Layer / purpose	Technology in implementation	Role in the project
Video resource processing	yt-dlp, youtube-transcript-api	Searches videos and retrieves available transcript segments.
Authentication	bcrypt, JWT, Google OAuth token verification	Password hashing, API identity tokens and Google sign-in.
Deployment support	Docker, Docker Compose, Render/Vercel configuration	Container and cloud deployment configuration included in the source.

The repository manifests and package files identify the above technologies. External keys such as `GEMINI_API_KEY`, OAuth client identifiers and database URLs must be provided through environment variables and must not be placed in the report or committed source. The Compose file configures a backend and PostgreSQL database; the web front end can be run with its package scripts. The student-facing frontend

is deployed at https://career-compass-s5.netlify.app/ [15]. This confirms the availability of the frontend entry point, but environment-specific APIs, authentication, external services and database connectivity must be validated separately; deployment configuration alone does not establish that every workflow is healthy.

3.7 EVALUATION METHODOLOGY

The career-classification notebook documents a five-fold StratifiedKFold evaluation with shuffling and a fixed random seed. Candidate algorithms are compared using mean accuracy, standard deviation of accuracy, macro precision, macro recall and macro F1-score. The selected model is then fitted on the full dataset and saved with the TF-IDF vectorizer and label encoder. Automated tests validate the API health route, role listing, valid skill submission, invalid input handling, uncertainty fallback and known role-to-skill pairings. [14].

A methodological caution is important: the notebook fits the TF-IDF vectorizer on the full dataset before creating the cross-validation folds, then cross-validates the classifier on the transformed features. This allows the vectorizer vocabulary and inverse-document-frequency information to observe the full corpus before validation. A stricter evaluation should put both vectorization and classification inside a scikit-learn Pipeline and fit the pipeline separately within each fold. The reported 78.86% result is reproduced as the repository's documented benchmark, but it should be re-evaluated with this cleaner protocol and an untouched test set before making

generalization claims.

3.8 Detailed Career Prediction Workflow

The career-analysis implementation followed a staged inference sequence. The request began with a non-empty text field containing a learner's technical skills. The backend validated the field, normalized it and passed the cleaned text to the fitted TF-IDF vectorizer. The resulting sparse feature vector was sent to the saved classifier. Where probability estimates were available, the service sorted the class scores and mapped the highest scores back to role labels through the saved label encoder. The result object included the top role, confidence, uncertainty flag, top-three candidates and role-specific skill-gap summaries.

The supplied backend code loaded model artefacts and the role-skill dataset when the application started. The loading function checked for the presence of the estimator, vectorizer, label encoder and data file, then constructed the role-skill vocabulary. This startup loading approach reduced repeated file access during prediction requests. Failure to locate any required artefact caused an explicit file-not-found error rather than silently returning an invented prediction. Table 3.A outlines the stages in the role-skill inference pipeline.

Table 3.A. Illustrative role-skill inference pipeline stages.

Stage	Input	Operation	Output / decision
1. Request validation	Skill text	Reject empty or invalid values; enforce schema	Validated skill string
2. Text cleaning	Validated string	Lowercase; replace non-alphanumeric punctuation with spaces; normalize whitespace	Cleaned text
3. Feature extraction	Cleaned text	Apply saved word-level TF-IDF features with unigram and bigram terms	Sparse feature vector
4. Role scoring	Feature vector	Run Logistic Regression probability estimates	Scores across 11 known roles
5. Candidate ranking	Class scores	Sort descending and decode labels	Up to three role candidates
6. Uncertainty check	Highest score	Compare top score with configured 0.40 threshold	Confidence/uncertainty indicator
7. Skill-gap report	Input tokens and role vocabulary	Calculate matched and missing tokens and coverage ratio	Role cards and skill-gap response

3.8.1 Input validation and normalization

The API schema required a `skills` value and rejected blank or whitespace-only input. The regression and API tests also included non-alphabetic input such as digits and

punctuation, and expected a validation error. In the backend implementation, the cleaning function returned an empty string for values that were not strings; for valid strings it converted characters to lowercase, replaced characters outside letters, digits and whitespace with spaces, collapsed repeated whitespace and trimmed both ends. This normalization reduced surface differences such as capitalization and punctuation before vectorization. It did not, however, perform synonym resolution, spelling correction or semantic matching between different skill names.

3.8.2 Candidate ranking and uncertainty threshold

The saved Logistic Regression estimator supported probability-based prediction. The implementation selected the indices with the highest scores, decoded them into role labels and returned the candidate score as a confidence value. The configured threshold was 0.40. If the highest score fell below that threshold, the response marked the prediction as uncertain while still exposing alternative candidates. The threshold was a software policy for presenting caution; it was not reported as a calibrated or experimentally optimized probability cutoff.

3.9 Dataset Structure and Class Coverage

The supplied career dataset contained two columns: `skills` and `job_role`. Its 440 rows were distributed evenly over eleven class labels, with forty examples per role. The repository files included a CSV copy for offline analysis and a copy under the backend directory. This balance supported comparisons between class labels within the dataset, but it did not establish that the examples represented the full population of students or current job advertisements. No verified employer-level job-vacancy feed was included in the inspected artefacts. Table 3.B lists the per-role distribution for the supplied dataset.

Table 3.B. Role-label distribution in the supplied dataset.

Role label	Records
AI Engineer	40
Backend Developer	40
Data Analyst	40
Data Engineer	40
Data Scientist	40
Database Administrator	40
DevOps Engineer	40
Frontend Developer	40
ML Engineer	40

Role label	Records
Mobile Developer	40
QA Engineer	40
Total	440

The role labels represent the classifier's available output classes. A user's skills could overlap across multiple labels; therefore, the implementation returned alternative candidates rather than treating the output as a definitive career decision. The report did not claim that the dataset provided salary, geographic demand, employer preferences or a time-based labour-market trend.

3.10 Text Feature Construction

The trained vectorizer represented each input profile as a weighted sparse feature vector. The inspected saved artefact used word-level features, a one-word and two-word n-gram range, a maximum of 250 features, smoothed inverse-document-frequency weights and L2 normalization. The supplied feature summary reported a 440 by 250 feature matrix. The weighting scheme reduced the influence of terms occurring broadly across documents and assigned more discriminative weight to terms that better distinguished profiles within the corpus.

The general form documented for the feature representation was:

$$TF\text{-}IDF(t, d) = TF(t, d) \times IDF(t); \quad IDF(t) = \log((1 + N) / (1 + DF(t))) + 1$$

The supplied artefact inspection recorded `sublinear_tf=False`, whereas the accompanying feature description mentioned `sublinear_tf=True`. The report retained the saved artefact's inspected setting as the basis for inference description and flagged the difference as a reproducibility issue. The vocabulary and inverse-document-frequency values were learned from the supplied corpus, so unfamiliar terms that did not enter the learned vocabulary could contribute little or no evidence to the classifier.

3.11 Model Selection and Evaluation Protocol

The model-comparison materials reported a five-fold stratified cross-validation procedure with shuffling and a fixed random seed. The candidate models included Logistic Regression, Random Forest, Gradient Boosting, XGBoost and LightGBM.

The comparison used mean accuracy, accuracy standard deviation, macro precision, macro

recall and macro F1-score. Logistic Regression was the selected classifier according to the documented mean accuracy and macro metrics. The current report retained those values in the Results and Discussion chapter and did not treat them as a measure of employment success or learner improvement.

The inspected notebook fitted the TF-IDF vectorizer on the complete data before cross-validation folds were applied to the transformed feature matrix. As a result, vocabulary and inverse-document-frequency information had access to examples that later appeared in validation folds. This is a methodological leakage risk. A stricter protocol would fit the vectorizer and classifier together inside a scikit-learn pipeline during each fold and reserve a held-out test set. The existing 78.86% benchmark was retained as a repository-reported baseline rather than presented as a leakage-free final estimate.

3.12 Skill-Gap Calculation Procedure

For each role, the backend constructed a vocabulary by collecting unique space-separated tokens from the `skills` strings belonging to that role. At prediction time, the entered skills were lowercased and split on spaces and commas. The matched set was the intersection of the learner tokens and the role vocabulary; missing tokens were those in the role vocabulary but absent from the supplied input. Coverage was calculated as the number of matched role-skill tokens divided by the total number of role-skill tokens, multiplied by 100.

$$\text{Coverage (\%)} = (\text{Number of matched role-skill tokens} / \text{Total role-skill tokens}) \times 100$$

This calculation was simple and traceable, but it counted tokens rather than distinct multi-word competencies and did not measure skill proficiency, recency, project evidence or certification. For instance, the supplied README showed a sample DevOps profile with four entered tokens and a reported coverage of 2.6% (4 out of 151 vocabulary tokens). This was a demonstration output from the source repository, not a separately measured user outcome. A high model score and a high coverage ratio were distinct values and should not be interpreted interchangeably.

3.13 Learning-Graph Generation and Validation

The supplied learning-path code requested a structured JSON graph for a learner-selected topic. The prompt specified six to eight nodes, foundational-to-advanced ordering, at least one prerequisite edge for every non-root node, initial recommendations for the first one or two concepts, and locked status for later concepts. Node fields included an identifier, label, description, beginner explanation, status, level, practice problems, related topics and resource metadata. The generated identifiers were translated into stored identifiers by the API layer before the interface displayed the graph.

The learning interface used a directed graph representation to show prerequisite relationships, allowed node selection and exposed concept details. The backend data models represented users, learning goals, nodes, directed edges, resources, progress, activity and practice completion. Resource limits in the supplied AI-service code capped retained resources by type: up to seven videos, five articles and three courses or tutorials per node. Those limits controlled list size; they did not independently verify the correctness, source quality or pedagogical suitability of the returned resources. Table 3.C summarizes the graph-generation constraints found in the supplied code.

Table 3.C. Learning-graph generation constraints recorded in the supplied code.

Element	Constraint evidenced in supplied code	Verification caution
Graph size	Six to eight nodes requested	Generated node count must be checked per response
Prerequisites	Every non-root node requested to have a prerequisite edge	Edge direction and missing prerequisites require validation
Initial status	First one or two nodes recommended; later nodes locked	Actual UI state depends on returned graph and stored progress
Videos	Maximum seven retained per node	Availability/transcript quality not guaranteed
Articles	Maximum five retained per node	Quality and relevance not guaranteed
Courses/tutorials	Maximum three retained per node	Resource labels need review

Generated graphs require review before being treated as a correct curriculum. A future validation layer should check schema completeness, node identifier uniqueness, edge references, cycles or permitted cycles, prerequisite ordering and resource integrity. The supplied evidence described generation and storage logic; it did not include a controlled expert evaluation of the correctness of generated graphs.

3.14 Learning Resources, Video Snippets and Practice

The resource service used video search and transcript tooling to locate candidate educational videos and retrieve transcripts where available. A generative service then selected a relevant continuous segment of approximately 60 to 90 seconds. Where transcripts could not be retrieved, the service provided a fallback response. In the practice workflow, the application could request practice items for a concept and return automated grading feedback and a model answer after a learner submitted a response.

These interactions were dependent on external API credentials, transcript availability, network access, rate limits and model responses. The report did not count successful code paths as proof that every external service was consistently available. The current artefacts also did not establish a human-labelled benchmark for grading quality or a measured improvement in learner outcomes. Automated feedback was therefore considered formative support rather than a replacement for instructor assessment.

3.15 API Boundaries, Persistence and Security

The supplied career-analysis backend exposed health, role-listing and prediction endpoints through a FastAPI service. The request and response schemas used typed validation. The frontend code was organized separately from the Python model service. The learning-path repository used FastAPI and SQLAlchemy models and configured PostgreSQL, with environment variables used for values such as database URLs and external API keys. The repository included authentication support based on password hashing and Google sign-in/token handling, along with configuration for containerized development.

API boundaries separated input validation, model execution, data persistence and external calls. That separation improved maintainability, but deployment configuration alone did not prove the health of every service in a hosted environment. Production readiness would require checks for CORS restrictions, access control, token protection, secret handling, error messages, rate limits, data retention and isolation between users. External API responses should be handled as untrusted content and displayed with appropriate validation and failure states.

3.16 Reproducibility and Evidence Limitations

The supplied career backend test runner was executed during validation of the supplied source files. It returned successful responses for the health endpoint, supported-role listing, valid prediction and input validation, and printed that all backend API tests passed. It also emitted scikit-learn version warnings: the serialized model artefacts reported creation under version 1.4.2 while the active runtime used version 1.8.0. The warnings did not prevent this test run, but the version mismatch should be resolved by pinning the supported environment or rebuilding the artefacts in a documented runtime.

The submitted source archive and the second learning-path archive use different repository names and application branding. The content and screenshots correspond to related functional ideas, but the archives alone do not prove that every module is fully integrated into one deployed CareerCompass AI user journey. This report therefore describes module-level functions separately and kept full cross-module integration, live demand aggregation and learning-outcome evaluation as unverified or future work.

The method and evidence status described above defined the boundary of the prototype. Reproducibility improvements, a leakage-free evaluation, a normalized skill taxonomy and module-level validation were necessary before claims about current labour-market demand, generalization to unseen users or measurable educational impact could be supported.

3.17 Test Design and Error Handling

CareerCompass AI was developed as a student-oriented prototype connecting a curated skill-profile classifier with an AI-assisted learning workspace. The career component represented 440 skill profiles across eleven roles and used text normalization, TF-IDF features and Logistic Regression to return ranked role candidates. The documented five-fold stratified benchmark reported 78.86% mean accuracy (standard deviation 1.54%), macro precision 0.8014, macro recall 0.7886 and macro F1-score 0.7833. These values describe the supplied repository’s evaluation protocol; TF-IDF was fitted before cross-validation, so a leakage-free re-evaluation remains necessary before stronger claims about generalization. The system also produced token-level matched/missing skill summaries and a coverage ratio, and the

learning-path code supported generation of

prerequisite graphs, concept resources, practice feedback and progress tracking. The available career API tests passed in the preparation environment, subject to scikit-learn version warnings. These checks did not validate a complete shared-user journey across both repositories. The skill-role dataset was static and curated rather than a verified live vacancy feed, and the coverage ratio measured vocabulary overlap rather than proficiency or job readiness. The project therefore demonstrated a feasible career-to-learning decision-support concept, but did not establish live skill-demand aggregation, employment outcomes or measured learning gains.

The successful career API test run in the preparation environment was accompanied by warnings about model artefacts saved with scikit-learn 1.4.2 and a runtime using version
1.8.0. A reproducible evaluation should pin the runtime to the artefact version or retrain and serialize the entire preprocessing-and-classification pipeline in a single controlled environment. The team should record dependency versions, dataset revision, random seed, evaluation split, test output and model-artifact checksum when reporting later benchmarks.

Finally, route-level tests and local component checks do not substitute for an end-to-end test with a shared user identity and persisted state across the career and learning applications. Such a test should cover a new user entering skills, reviewing alternatives, selecting a target topic, generating the graph, completing prerequisites, accessing learning resources, submitting practice answers and viewing progress. Failures should be recorded by step so that future integration claims can be linked to reproducible evidence.

Summary. The methodology combines a supervised TF-IDF and Logistic Regression classifier with a separate generative-AI learning service. It documents the curated dataset, role-vocabulary coverage, service boundaries, persistence, error handling and evaluation procedure. The reported benchmark requires leakage-free re-evaluation, and career-to-learning integration still needs end-to-end testing.

CHAPTER - IV

PROPOSED WORK MODULES

This chapter describes the main functional modules and the data flow between them. The career module transforms a short skill profile into role hypotheses and a token-based gap report. The learning module creates a graph of concepts and connects it to resources, practice and learning progress. The modular description follows the functional organization present in the source code without claiming that a single endto-end user session has already been validated across all modules.

4.1 CAREER PROFILE AND ROLE PREDICTION MODULE

The skill-input interface accepts a comma-separated or space-separated string of technical skills. The API validates that the field is present and within the configured input length, trims surrounding whitespace and rejects inputs that contain no letters. At application startup, the model service loads the serialized Logistic Regression estimator, TF-IDF vectorizer, label encoder and CSV dataset. Loading at startup avoids reloading the artefacts for each request.

For a valid input, the service applies text cleaning, vectorizes the cleaned string and obtains the predicted probabilities for the 11 known classes. It orders the class scores, maps class indices back to role names and constructs a response with the top predicted role, confidence, uncertainty flag, threshold and topthree candidate objects. Each candidate includes matched and missing skills, required vocabulary size and coverage. This response is suitable for rendering as ranked cards in a web interface. Figure 4.1 shows the skill-entry interface, while Figure 4.2 shows ranked predictions with confidence and skill-gap details.

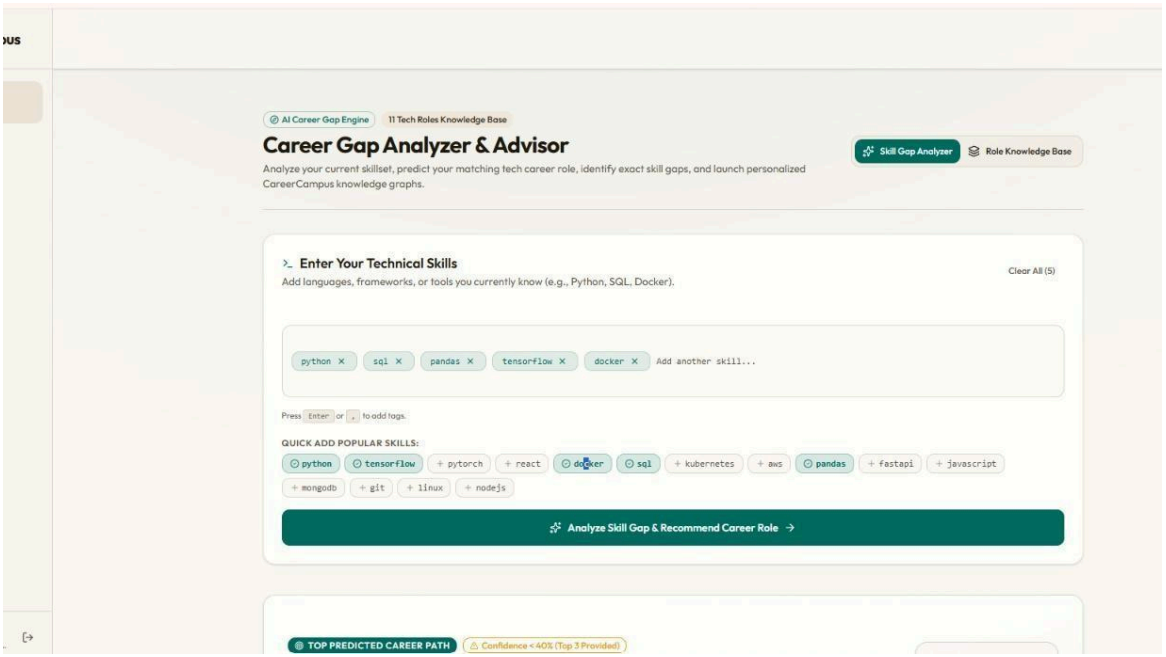


Figure 4.1. Career skill-input interface with selected technical skills and the analysis action.

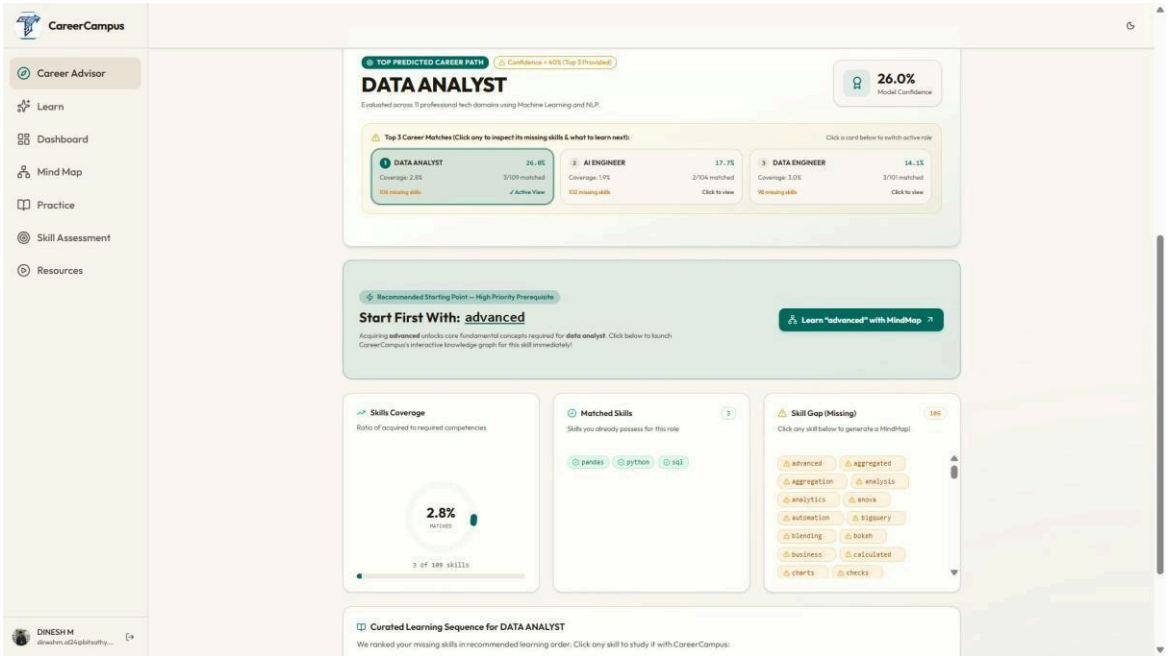


Figure 4.2. Predicted career roles with model confidence, skill coverage, matched skills and missing skills.

4.2 SKILL GAP AND COVERAGE ANALYSIS MODULE

The gap module builds one skill vocabulary per role from the role examples in the dataset. The service compares the learner's token set against the vocabulary for each of the top three candidate roles. The output distinguishes terms that match from terms that are absent and calculates coverage for each candidate. This allows a student to inspect trade-offs: the role with the highest classifier confidence may not necessarily have the

highest vocabulary coverage, and different candidate roles can require different next learning steps.

The coverage implementation uses a set intersection and set difference, which are efficient for short skill inventories. It also has limitations. First, word tokenization can treat "machine learning" as two terms rather than one competency. Second, punctuation and normalization are not completely identical between the classifier's `clean_text` path and the gap-calculation path. Third, the role vocabulary is derived from dataset tokens, including terms that may represent tools, subfields, acronyms or incidental context rather than standardized skills. These limitations are reported so the number is not misinterpreted as a validated employability score. Figure 4.3 shows the career-advisor interface and access to the supported-role knowledge base.

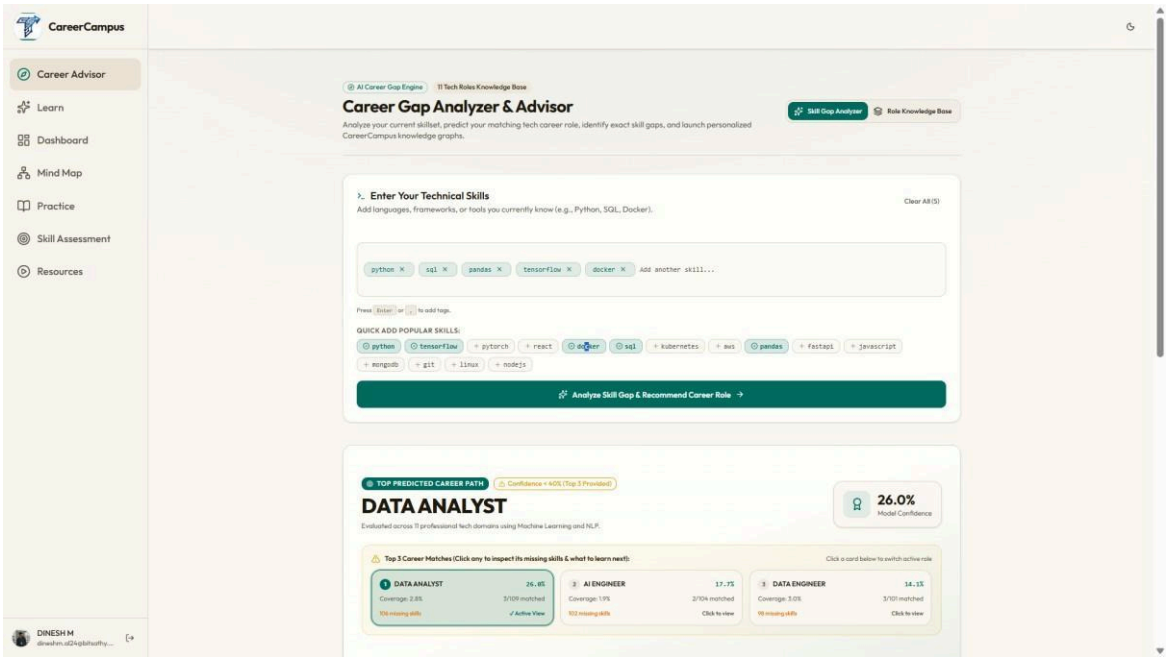


Figure 4.3. Career advisor interface with access to the supported-role knowledge base.

4.3 PERSONALIZED KNOWLEDGE GRAPH MODULE

The learning module allows the student to describe a topic they want to learn, such as "machine learning fundamentals" or "backend development." The graph-generation request instructs the Gemini model to create a small directed graph ordered from foundational to advanced concepts. Nodes contain an identifier, label, brief description, beginner-oriented explanation, status, level, practice prompts, related topics and resource titles/types. Edges encode prerequisite-to-dependent relationships.

The front end uses React Flow to render an interactive canvas with pan, zoom and drag behaviours. Concept statuses are visually distinguished so the learner can understand what is completed, available, weak or locked. Selecting a node opens its detail view, which presents the concept explanation, related topics and learning actions. Node state is not merely decorative: the backend checks prerequisites and combines stored completion information when returning graph nodes.

The generator's target of six to eight nodes keeps the initial graph compact enough for a learner to inspect. However, graph breadth and depth are still driven by generative output, and a short graph may omit important prerequisites for broad topics. The graph should therefore be treated as an adaptive starting plan. For advanced or safety-critical topics, it should be compared with a trusted syllabus or reviewed by a subject expert. Figure 4.4 illustrates the prerequisite graph and concept-detail view.

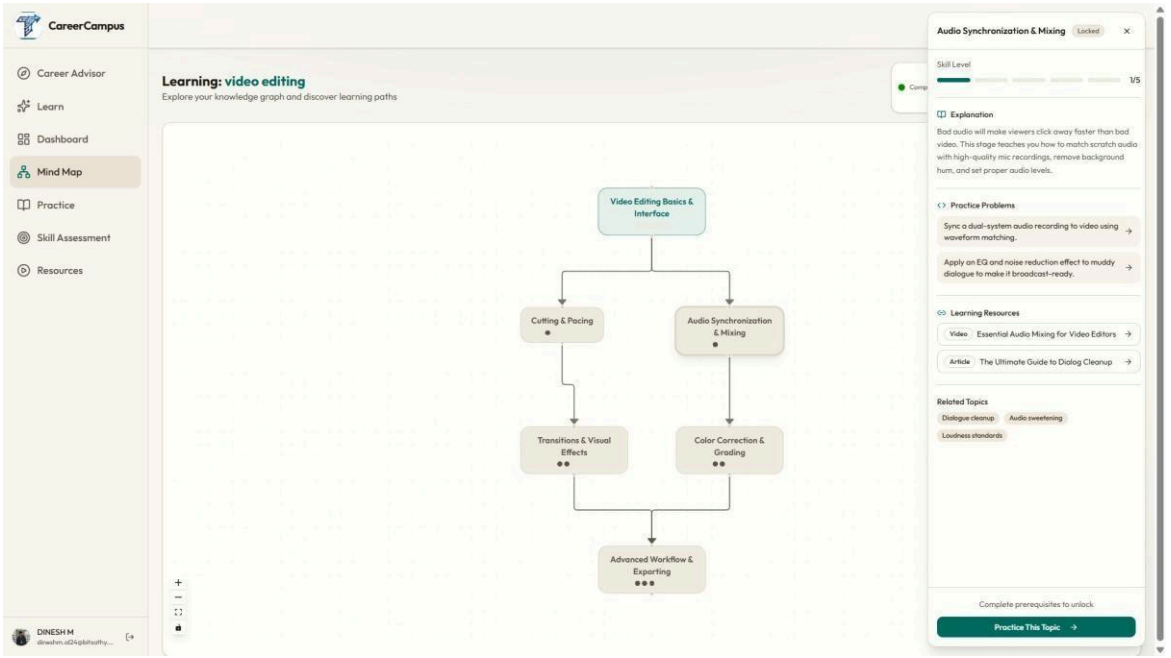


Figure 4.4. Interactive learning knowledge graph showing prerequisite relationships and concept details.

4.4 LEARNING PROGRESS AND RECOMMENDATION MODULE

The progress service records which concept nodes a user has completed. It can mark a concept as started, completed or weak, and can reverse a completion state. A separate activity table stores the day on which a user completed at least one node, allowing the dashboard to derive a learning streak. Practice completion records identify the concept and problem index, preventing duplicate completion records for the same item.

The dashboard and recommendation pages summarize progress for the active topic. Their functions include overall completion percentage, number of completed concepts, learning streak, knowledge gaps and a radar chart of concept levels. The recommendation service orders possible next concepts using node status and prerequisite state; quick-win items are presented as concepts estimated to require a shorter study session. Such time estimates should be treated as guidance because the code does not measure every student's actual study duration. The separate Skill Assessment view summarizes concept completion, average skill level and recommended next topics. The supplied screenshot shows the learner-selected sample topic "video editing"; it demonstrates topic-generic progress tracking rather than a role-specific assessment.

Multi-topic learning goals enable learners to create a separate graph for each topic, switch the active goal, view previous goals and delete a goal. This is useful when a student wants to explore related technical areas or study a short list of independent topics. In the current implementation, continuity depends on the user identifier and database state; an integrated cross-module account journey should be validated so the same learner does not experience fragmented histories. Figure 4.5 shows the learning dashboard, progress indicators and next-topic recommendations.

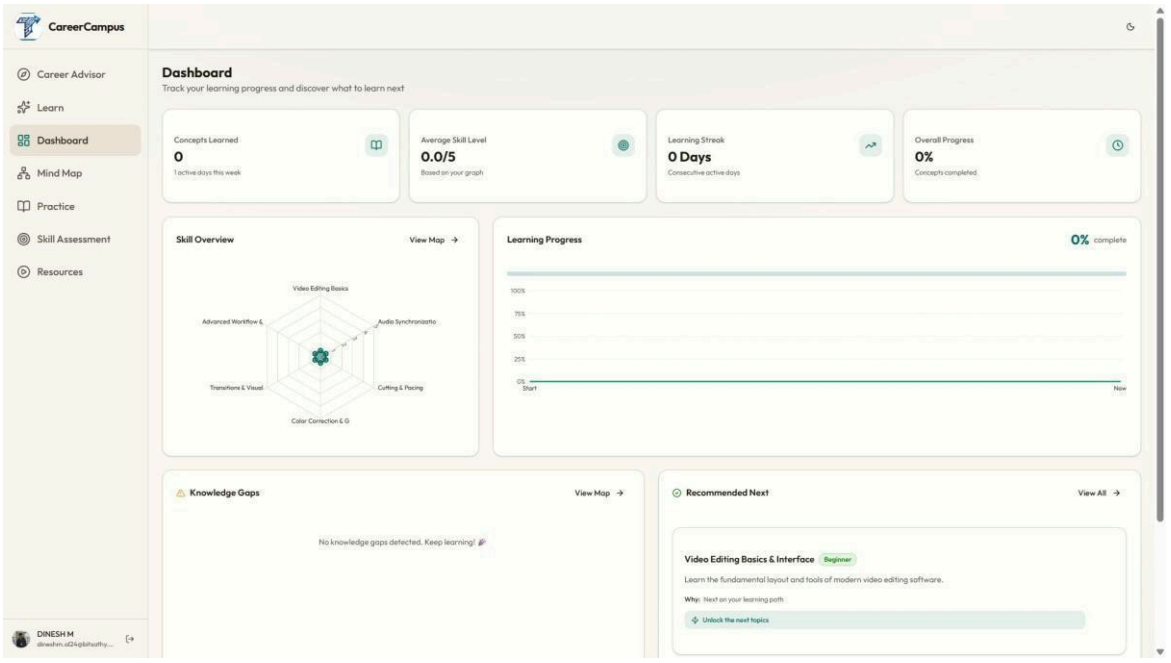


Figure 4.5. Learning dashboard showing progress indicators, skill overview and recommended next topics.

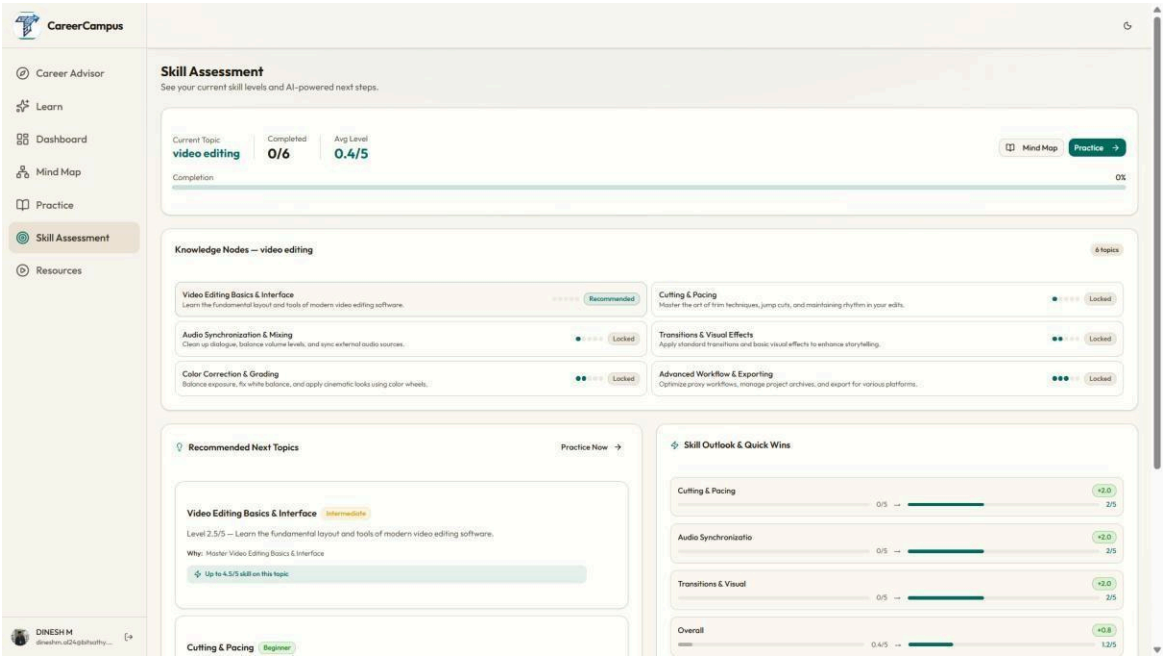


Figure 4.6. Skill Assessment view for the selected "video editing" topic, showing completion status, average level and recommended next topics.

4.5 RESOURCE, VIDEO SNIPPET AND PRACTICE MODULES

The resource view groups material by learning topic and type. Graph nodes store resource title/type metadata, while the resource-service layer can help retrieve a relevant educational video. The YouTube workflow searches for candidate videos, attempts to fetch English transcript segments through the transcript API, converts those segments into timestamped text and requests that Gemini select a continuous excerpt of approximately 60 to 90 seconds. The response can include the video title, channel, start and end timestamps, URL and a short explanation of why the selected segment is relevant. If candidates have no usable transcript, the service returns a fallback rather than guaranteeing a snippet.

Practice problems can be generated or attached to concepts. The learner submits an answer through the grading API, which asks the language model to assess the response and return a score, explanatory feedback and a model answer. The feedback is intended to help the student understand the difference between their response and a reference answer. This is an educational assistance function, not a formally validated exam grader; rubric design, consistency across equivalent answers and human review remain future evaluation needs. Figure 4.7 shows resource discovery and a video snippet; Figure 4.8 shows practice feedback.

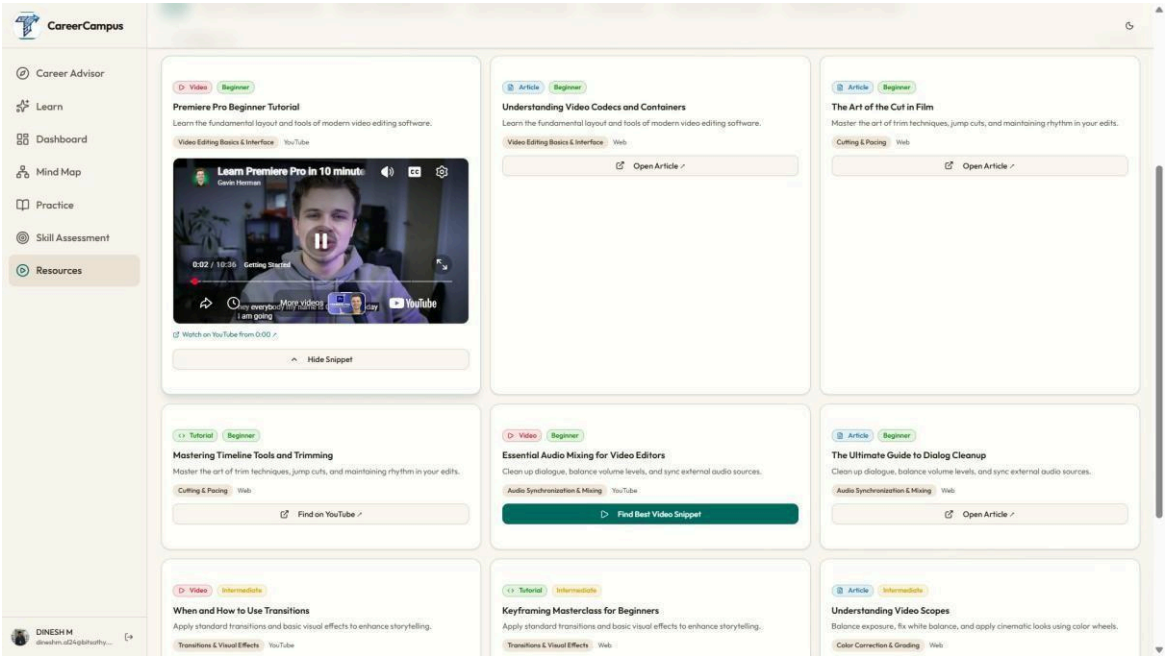


Figure 4.7. Resource explorer displaying learning resources and an educational-video snippet.

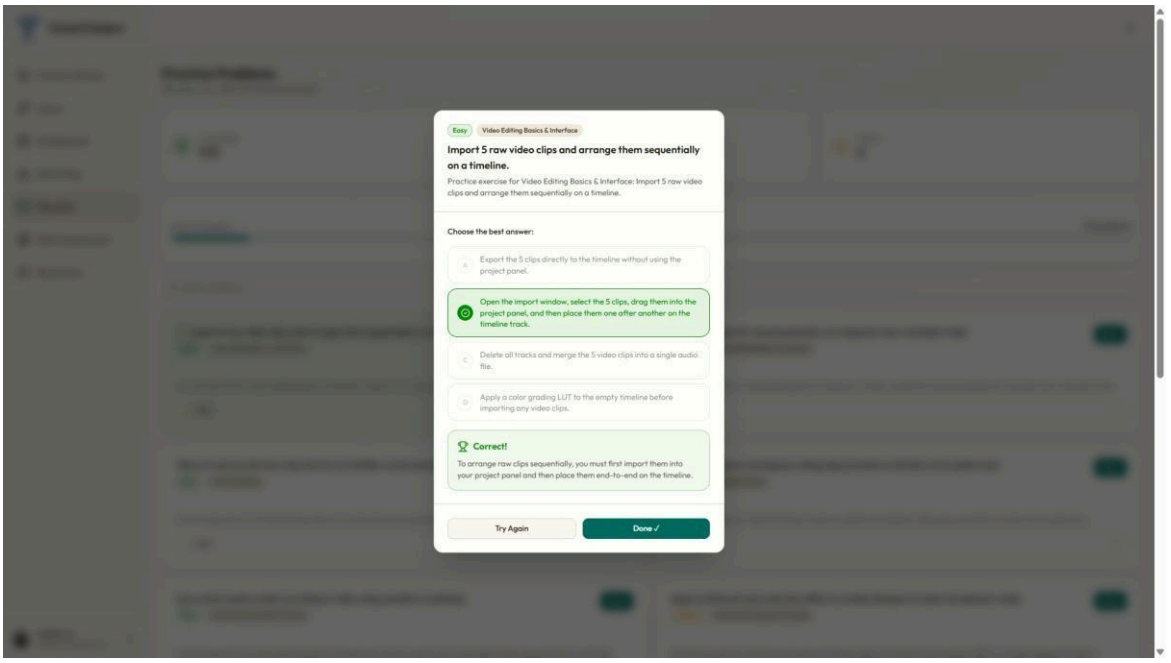


Figure 4.8. Practice interaction showing answer feedback and the correctness explanation.

4.6 USER MANAGEMENT AND DATA PERSISTENCE

The learning service includes a User entity with an identifier, email, display name, avatar, password hash or Google identity, and creation timestamp. Email/password authentication hashes passwords using bcrypt and issues JSON Web Tokens after successful login. The Google sign-in flow validates the received ID token and checks its audience against the configured client identifier. The configuration requires

environment-specific secrets and correct alignment between the front-end client ID and the back-end audience check.

The learning database represents goals, concepts, prerequisite edges, resources and progress in relational tables. A learning goal belongs to a user and has related nodes; an edge stores the source and target

identifiers; a node resource stores its title and type; user progress associates a user with a completed node; user activity tracks days with learning activity; and practice-completion rows record completed problem indices. PostgreSQL is configured for containerized use, while a local SQLite fallback supports lightweight development.

4.7 API DESIGN AND DATA MODEL

The career-analysis API uses three primary endpoints: `/health` for service readiness, `/roles` for the role knowledge base, and `/predict` for predictions and skill-gap data. Request and response models are validated with Pydantic. The response structure contains a predicted role, confidence, uncertainty flag, threshold, matched skills, missing skills, coverage and three candidate role objects. Table 4.1 summarizes career API endpoints, while Table 4.2 lists the principal learning-data entities.

Table 4.1. Primary API endpoints and response summary.

API endpoint	Method	Purpose	Request / response summary
/health	GET	Service readiness	Returns health, model-loaded flag and API version.
/roles	GET	Role knowledge base	Returns all supported role labels and each role's prerequisite token list.
/predict	POST	Career recommendation	Accepts `skills`; returns ranked roles, confidence, matched/missing skills and coverage.
/api/generate-graph	POST	Create learning path	Accepts user/topic information; requests and stores a knowledge graph.
/api/mindmap/{user_id}	GET	Retrieve graph	Returns graph nodes and prerequisite edges for the active or selected goal.
/api/progress/complete	POST	Complete concept	Records completion and daily activity.
/api/dashboard/{user_id}	GET	Learning summary	Returns progress and activity summaries for the dashboard.
/api/recommendations/{user_id}	GET	Next-step suggestions	Returns prioritized concepts and quick-win suggestions.

/api/grade-answer	POST	Grade practice response	Returns AI-generated assessment feedback and model answer.

API endpoint	**Method**	**Purpose**	**Request / response summary**
/api/snippet/{node_id}	GET	Find video segment	Returns transcript-grounded segment metadata when available.

Table 4.2. Learning data entities and their purposes.

Entity/table	**Important fields**	**Purpose**
users	id, email, name, avatar_url, hashed_password, google_id, created_at	Account and authentication identity.
learning_goals	id, user_id, goal, created_at	Stores each topic requested by a user.
nodes	id, user_id, goal_id, label, status, level, coordinates, description, explanation, practice_prompts, related_topics	Stores each learning concept and its user-visible details.
edges	source_id, target_id	Represents prerequisite links between nodes.
node_resources	node_id, title, type	Stores resource metadata associated with a concept.
user_goals	user_id, goal, goal_id	Tracks the currently active learning goal.
user_progress	user_id, node_id	Tracks completion of nodes.
user_activity	user_id, activity_date	Records active learning dates for streak calculation.
user_practice_completion	user_id, node_id, problem_index	Tracks completed practice prompts.

Summary. Modules are documented; full end-to-end integration remains unverified.

CHAPTER - V

RESULTS AND DISCUSSION

The results in this chapter distinguish three forms of evidence: quantitative benchmark metrics for the career classifier, automated API and regression tests, and code-level review of the learning-path modules. The first two have reproducible numeric results within the supplied environment. The learning graph, AI content generation and full cross-module experience are described from implementation inspection; they should not be assigned accuracy, latency or usability statistics that have not been measured. Table 5.1 records the core dataset characteristics.

5.1 DATASET AND MODEL RESULTS

5.1.1 Dataset characteristics

Table 5.1. Dataset characteristics.

Characteristic	**Observed value**
Number of records	440
Number of input columns	2: skills, job_role
Missing values in inspected CSV	0 in either column
Supported role classes	11
Samples per class	40
Class balance	Equal class count across all roles
Unique skill strings	439 of 440 rows
Validation approach documented in notebook	5-fold stratified cross-validation, shuffle enabled, random_state=42

The dataset is balanced by design: each of the eleven role labels has forty skill-profile examples. This balance prevents large classes from dominating the reported overall accuracy, but it does not guarantee that examples are representative of real students or job vacancies. The inspected dataset has 439 unique skill strings across 440 rows, indicating that only one text profile is duplicated exactly; semantically similar or overlapping skill lists may still appear across roles.

5.1.2 Candidate model comparison

The model-training artefacts compare five candidate algorithms using the same documented stratifiedfold partitions. Logistic Regression achieved a mean accuracy of 78.86% with a standard deviation of 1.54 percentage points and the highest reported

macro F1-score among the evaluated candidates. Random Forest achieved 75.91% mean accuracy, Gradient Boosting 70.23%, XGBoost 65.00% and LightGBM 25.00%. The result supports selecting Logistic Regression as the current baseline for this constructed dataset; it does not demonstrate superiority over algorithms that were not evaluated or establish external validity. Table 5.2 reports the benchmark metrics, and Figure 5.1 visualizes mean accuracy across the folds.

Table 5.2. Classifier benchmark results under five-fold stratified cross-validation.

Model	Mean accuracy	Std. dev.	Macro precision	Macro recall	Macro F1
Logistic Regression (selected)	78.86%	1.54%	0.8014	0.7886	0.7833
Random Forest	75.91%	2.82%	0.7827	0.7591	0.7536
Gradient Boosting	70.23%	2.63%	0.7531	0.7023	0.7052
XGBoost	65.00%	7.02%	0.6859	0.6500	0.6457
LightGBM	25.00%	4.60%	0.2507	0.2500	0.1961

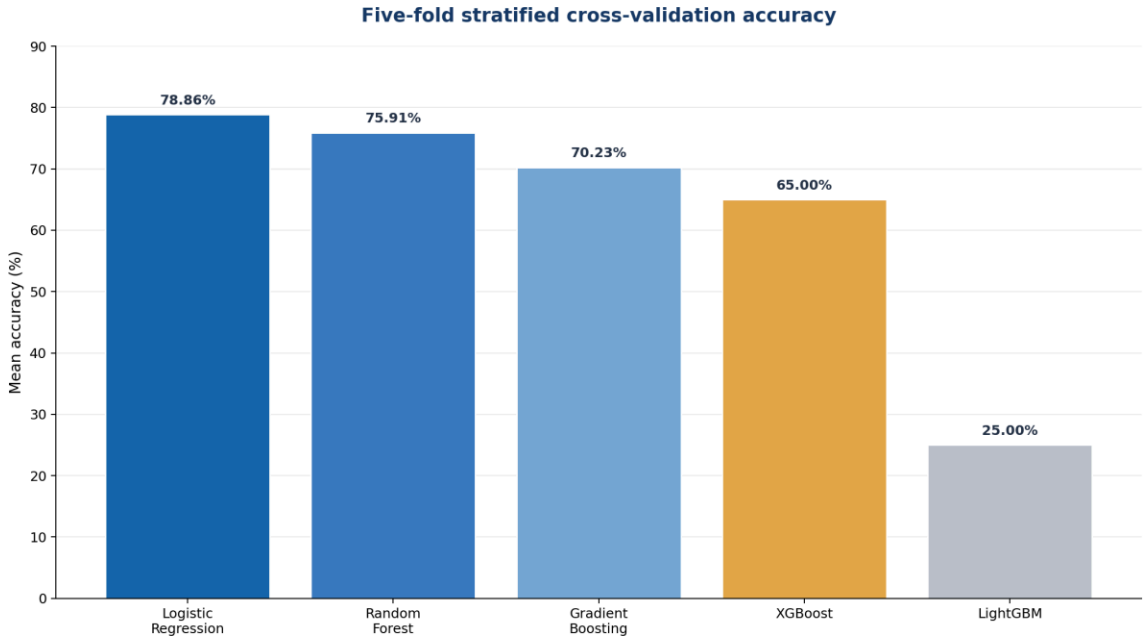


Figure 5.1. Mean classifier accuracy recorded in the project's five-fold stratified cross-validation report.

The selected model's macro precision of 0.8014 indicates that, averaged equally across classes, predicted role labels were relatively precise in the reported folds. Macro recall of 0.7886 summarizes the proportion of each class retrieved, averaged across classes. Macro F1 of 0.7833 combines these two metrics. The gap between accuracy and macro F1 is modest, but the full class-wise confusion matrix should be examined before discussing which career categories are confused most often. No such class-wise matrix is included in the supplied metrics file.

5.1.3 Interpretation of uncertainty and skill coverage

The API sets an uncertainty flag when the top role probability falls below 0.40. In this case the system still reports the three candidate roles and their gap summaries, which is preferable to silently treating a weak prediction as definitive. The top three role view is particularly relevant where skill sets overlap across positions. Coverage then gives a second, independent descriptive signal. A high classifier probability and a high role-token coverage are not interchangeable, and neither implies that the student has demonstrated the skills through practical work.

The dataset and saved model artefacts were inspected, and the career API tests passed in the active runtime. The model artefacts were serialized with scikit-learn 1.4.2 and were loaded in the current environment using scikit-learn 1.8.0, which emitted compatibility warnings. Although tests passed, serialized estimator compatibility should be resolved by pinning a compatible runtime or retraining and re-exporting the model under the deployment version.

5.2 AUTOMATED TEST RESULTS

The following results were obtained by executing the supplied career-analysis backend test suite with `pytest -q tests`. All 17 test cases passed. Eight warnings were emitted because the serialized scikit-learn estimator objects were created under version 1.4.2 and loaded under version 1.8.0. The warnings do not negate the observed test pass, but they indicate a reproducibility risk that should be addressed before deployment. Table
5.3 summarizes the outcomes of the automated career API tests.

Table 5.3. Automated career API test results.

Test area	Checks represented in suite	Result
Health endpoint	Returns HTTP 200, healthy status, model-loaded flag and API version.	PASS
Role endpoint	Returns exactly 11 role entries with prerequisite skill lists.	PASS
Valid prediction	Known AI-engineering skill profile returns the expected role and three candidates.	PASS
Input validation	Rejects whitespace-only input and numeric/punctuation-only input with validation errors.	PASS

Uncertainty handling	Vague skill input returns a response with uncertainty state and top- three candidates.	PASS
Known role regression	11 curated role/profile pairs map to their expected role categories.	PASS

Test area	Checks represented in suite	Result
Total automated tests	API and model-regression tests executed in this run.	17 / 17 passed
Environment note	Scikit-learn pickle compatibility warnings: artefacts from 1.4.2 loaded using 1.8.0.	ACTION REQUIRED

These tests verify API contracts and selected expected model mappings; they do not constitute a comprehensive clinical, pedagogical or workforce study. They do not validate whether the selected role is suitable for an individual, whether the skill coverage predicts employment success, or whether learning graph recommendations improve learning outcomes. No automated test suite was found in the learning service source at the time of inspection, so corresponding graph generation, prerequisite state, AI grading and video snippet behaviour require additional test cases.

5.3 FUNCTIONAL REVIEW OF LEARNING MODULES

A source-code review confirms that the learning service defines routes for goal creation and switching, graph generation and retrieval, node detail, completion updates, dashboard and recommendations, resource and practice retrieval, practice completion, answer grading and video snippets. The database model contains entities for users, learning goals, nodes, edges, resources, progress, activity and practice completion.

These

code-level findings establish that the functions are represented in the implementation; because no full system test was executed for the learning service in this report, they should not be reported as independently measured performance results. Table 5.4 provides the module-by-module verification matrix.

Table 5.4. End-to-end verification matrix for the learning modules.

Scenario	Expected validation	Status for this report
Create a learning goal	Submit topic; store 6-8 graph nodes and prerequisite edges; display graph.	Implemented path identified in code; end-to-end run pending.
Prerequisite unlocking	Keep a node locked until required prerequisite nodes are completed.	Logic identified in backend; automated tests pending.

Progress and streak	Record completion and activity date; update dashboard figures.	Data entities/routes identified; end-to-end tests pending.
AI answer grading	Return score, feedback and model answer for a submitted response.	Endpoint identified; response quality evaluation pending.
YouTube snippet selection	Search candidates, retrieve available transcript and select a 60-90 second segment.	Workflow identified; reliability and timestamp accuracy tests pending.
Switch learning goals	Load the graph/progress associated with the selected goal.	Routes and persistence identified; multi- user integration test pending.

Scenario	Expected validation	Status for this report
Unified career-to- learning route	Use target role/gap output to populate or suggest a corresponding learning goal.	Integration and shared- session behaviour require validation.

5.4 STRENGTHS AND LIMITATIONS

5.4.1 Strengths

The classifier is small enough to load as a local artefact and return top-three candidate roles with probabilities.

The role-skill comparison produces visible matched and missing items rather than a role label alone.

The knowledge-graph view helps learners interpret prerequisites and identify a next concept.

Progress, daily activity and practice-completion records provide a base for persistent learning summaries.

The API uses request/response validation schemas, and the career predictor has an automated 17-test suite that passed in this run.

The resource and grading workflows can reduce the manual effort required to assemble practice content and find relevant short video segments. Table 5.5 consolidates documented strengths, limitations and recommended responses.

5.4.2 Limitations and risks

Table 5.5. Strengths, limitations and recommended responses.

Issue	Evidence / implication	Recommended response
Static skill dataset	440 skill strings with 40 examples per role; data provenance is not linked to dated job ads.	Add documented job-posting sources, date metadata, deduplication and periodic refresh.

CV preprocessing leakage risk	Notebook fits TF-IDF on the full dataset before splitting indices into folds.	Use a Pipeline so vectorizer is fitted within each training fold; repeat cross-validation and keep a holdout set.
Token-level skill gap	Role skills are constructed from space- separated tokens, not normalized competency entities.	Create an explicit skill taxonomy with phrases, synonyms, levels and evidence fields.
Confidence interpretation	Threshold 0.40 is configured in code; no calibration analysis is present.	Evaluate calibration and choose threshold from validation criteria.
Model artefact version mismatch	Runtime emitted scikit-learn 1.4.2-to-1.8.0 loading warnings.	Pin the compatible library version or retrain and export under the target runtime.
Generated content quality	Language model output can be factually wrong or contain questionable prerequisite edges.	Add schema validation, link checks, curated references and human review.

Issue	**Evidence / implication**	**Recommended response**
Video transcript dependence	Snippet selection requires an available transcript and relies on external services.	Test language coverage, missing transcripts, timestamps and fallback behaviour.
Integration evidence	No full end-to-end test has been run across the career and learning journey.	Add shared navigation/session tests and verify selected career targets become coherent learning plans.
Security configuration	Career API CORS is permissive in the current source; external keys are environment-dependent.	Restrict allowed origins, rate-limit endpoints, protect secrets and review identity boundaries.
Market-demand claim	Automated live job-posting aggregation is not established by current source code.	Implement and evaluate the ingestion/ranking pipeline before claiming live demand analytics.

5.5 COST-BENEFIT ANALYSIS

The project is based predominantly on open-source development tools, which lowers the entry cost for a student prototype. React, Next.js, FastAPI, scikit-learn, SQLAlchemy, PostgreSQL and the graphrendering library have accessible community documentation. Local development can use the CSV data and serialized model artefacts without requiring a paid hosted database. The learning-service design uses an external generative-AI API and YouTube transcript/search tooling; these add quota, rate-limit, availability and potential usage-cost considerations. The qualitative comparison is presented in Table 5.6.

Table 5.6. Qualitative cost-benefit analysis.

Cost / benefit dimension	**Qualitative assessment**
Software licensing	Core development stack is predominantly open source; check each dependency licence before redistribution.

Cost / benefit dimension	**Qualitative assessment**
Infrastructure	A small prototype can be developed locally; hosted UI/API/database deployments may incur usage or hosting costs.

Generative AI	Graph generation, grading and snippet selection depend on API availability and usage quotas; budget must be reviewed before public deployment.
Maintenance	Requires dataset refresh, model revalidation, dependency updates, API key management and resource-link checks.
Student benefit	Can centralize role exploration, missing-skill review, topic sequencing, practice and progress visibility.
Faculty / placement benefit	May support structured mentoring conversations when its limits are explained; it should not replace placement judgement.
Risk cost	Poor recommendations may misdirect study time; confidence notices, human review and transparent limitations reduce this risk.

Summary. Entry costs remain low; ROI, learning gains and full integration remain unmeasured.

CHAPTER - VI

CONCLUSIONS AND SUGGESTIONS FOR FUTURE WORK

6.1 Conclusion

CareerCompass AI was developed as a student-oriented prototype connecting a curated skill-profile classifier with an AI-assisted learning workspace. The career component represented 440 skill profiles across eleven roles and used text normalization, TF-IDF features and Logistic Regression to return ranked role candidates. The documented five-fold stratified benchmark reported 78.86% mean accuracy (standard deviation 1.54%), macro precision 0.8014, macro recall 0.7886 and macro F1-score 0.7833. These values describe the supplied repository's evaluation protocol; TF-IDF was fitted before cross-validation, so a leakage-free re-evaluation remains necessary before stronger claims about generalization.The system also produced token-level matched/missing skill summaries and a coverage ratio, and the learning-path code supported generation of prerequisite graphs, concept resources, practice feedback and progress tracking. The available career API tests passed in the preparation environment, subject to scikit-learn version warnings. These checks did not validate a complete shared-user journey across both repositories. In addition, the skill-role dataset was static and curated rather than a verified live vacancy feed. The coverage ratio measured vocabulary overlap, not proficiency or job readiness. The project therefore demonstrated a feasible career-to-learning decision-support concept, but did not establish live skill-demand aggregation, employment outcomes or measured learning gains.

6.2 Suggestions for Future Work

Future work should begin with traceable collection of live job postings, including source attribution, retrieval timestamps, duplicate removal, skill normalization and a scheduled refresh process. Any reported demand should be limited to the sources, regions, roles and time windows actually observed.

The raw token-coverage calculation should be replaced by a reviewed skill taxonomy covering synonyms, multiword competencies, proficiency levels and evidence of practical ability, with role requirements validated against dated employer sources. The classifier should be re-evaluated with vectorization and classification placed inside one scikit-learn Pipeline and fitted independently within each cross-validation fold. An untouched test set, per-class metrics and error analysis should be added. Dependency versions should be pinned, or the model artefacts should be retrained and serialized in a documented environment; the saved TF-IDF parameter discrepancy should be reconciled and dataset and model revisions recorded.Further work should validate the integrated career-to-learning journey, persistent account state, graph prerequisites, generated resources, transcript handling and practice feedback under both successful and failed requests. Generated explanations, graph correctness and automated grading should be assessed through human review and a documented rubric, followed by user studies before claiming improvements in learning effectiveness. Security, privacy, access control, secret management, logging, external-service failure handling, mobile usability and deployment monitoring should also be strengthened before broader use.

Summary. In summary, CareerCompass AI connects role classification and vocabulary-based skill-gap reporting with an AI-assisted learning workspace. Its current value is educational decision support rather than verified job-market forecasting, hiring eligibility assessment or a guarantee of employment outcomes.

REFERENCES

[1] World Economic Forum, “The Future of Jobs Report 2025,” Geneva, Switzerland, Jan. 2025. [Online]. Available: https://www.weforum.org/publications/the-future-of-jobs-report-2025/

[2] A. Patil, D. Suwalka, A. Kumar, G. Rai, and J. Saha, “A Survey on Artificial Intelligence (AI) based Job Recommendation Systems,” in Proc. 2023 International Conference on Sustainable Computing and Data Communication Systems (ICSCDS), Erode, India, Mar. 2023, pp. 730-737, doi: 10.1109/ICSCDS56580.2023.10104718.

[3] Y. Fettach, A. Bahaj, and M. Ghogho, “JobEdKG: An uncertain knowledge graph-based approach for recommending online courses and predicting in-demand skills based on career choices,” Engineering Applications of Artificial Intelligence, vol. 131, Art. no. 107779, 2024, doi: 10.1016/j.engappai.2023.107779.

[4] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513-523, 1988, doi: 10.1016/0306-4573(88)90021-0.

[5] Scikit-learn developers, “TfidfVectorizer,” scikit-learn API documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.ht ml

[6] Scikit-learn developers, “LogisticRegression,” scikit-learn API documentation. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html

[7] C. Liu, H. Zhang, J. Zhang, Z. Zhang, and P. Yuan, “Design of a learning path recommendation system based on a knowledge graph,” International Journal of Information and Communication Technology Education, vol. 19, no. 1, pp. 1-18, 2023, doi: 10.4018/IJICTE.319962.

[8] H. Ngo, K. Vo, and T. Nguyen, “Personalized Learning Path Recommendations: Fusing Knowledge Graph Embedding, Sequence Mining, and Collaborative Filtering,”

in Proc. 2024 IEEE International Conference on Big Data (BigData), Washington, DC, USA, Dec. 2024, pp. 8145-8153, doi: 10.1109/BigData62323.2024.10825001.

[9] A. Li, Y. Li, and X. Gao, “Personalized learning path recommendation based on knowledge graphs: A survey,” Electronics, vol. 15, no. 1, Art. no. 238, Jan. 2026, doi: 10.3390/electronics15010238.

[10] Google, “Structured outputs,” Gemini API documentation. [Online]. Available: https://ai.google.dev/gemini-api/docs/structured-output

[11] FastAPI, “First steps - interactive API documentation,” FastAPI documentation. [Online]. Available: https://fastapi.tiangolo.com/tutorial/first-steps/

[12] Vercel, “Next.js documentation,” Next.js documentation. [Online]. Available: https://nextjs.org/docs

[13] React Flow contributors, “React Flow: node-based UIs in React,” documentation. [Online]. Available: https://reactflow.dev/

[14] Project implementation artefacts supplied for CareerCompass AI: skill-profile dataset, model card, metric report, API source, frontend source, learning-path API, database models and AI/video services, accessed Oct. 2026.

[15] CareerCompass AI, “Student-facing frontend deployment.” [Online]. Available: https://career-compass-s5.netlify.app/ (accessed 11 Oct. 2026).

APPENDIX I

SELECTED CODE EXCERPTS

The excerpts below reproduce small portions of the supplied source code to document key operations.

Listing II.1. Text normalization from the supplied career backend.

```
def clean_text(text: str) -> str:
    if not isinstance(text, str):
        return ""
    text = text.lower()
    text = re.sub(r"[^a-zA-Z0-9\s]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

Listing II.2. Candidate ranking from the supplied career predictor.

```
proba = self.model.predict_proba(X)[0]
top_indices = np.argsort(proba)[::-1][:top_k]
results = []
for idx in top_indices:
    role_name = self.label_encoder.inverse_transform([idx])[0]
    conf = float(proba[idx])
    results.append({"role": role_name, "confidence": conf})
return results
```

Listing II.3. Vocabulary coverage and threshold logic.

```
coverage = (
    len(matched_skills) / required_skills_count * 100
    if required_skills_count > 0 else 0.0
)
is_uncertain = confidence < self.confidence_threshold
```

Listing II.4. Resource caps declared in the supplied learning-path service.

```
MAX_VIDEOS_PER_NODE = 7
MAX_ARTICLES_PER_NODE = 5
MAX_COURSES_PER_NODE = 3

# The learning-graph prompt requests 6 to 8 nodes,
# prerequisite edges, initial status and practice/resource data.
```

APPENDIX II

SUPPLEMENTARY TEST MATRIX

The following test cases are present in the supplied career backend tests.

Test area	Test expectation in supplied tests	Evidence / outcome
Health endpoint	HTTP 200; healthy status; model loaded true; version 1.0.0	Pass in supplied test runner
Role listing	Eleven role records with role, skill count and prerequisite-skill fields	Pass in supplied test runner
Valid prediction	Known Python/deep-learning skills map to AI Engineer; three candidates returned	Pass in supplied test runner
Empty input	Whitespace-only value returns HTTP 422	Pass in supplied test runner
Invalid input	Digits/punctuation-only value returns HTTP 422	Covered by pytest source
Low-confidence input	Uncertain flag and top-three candidates returned	Covered by pytest source
Known role regression	Eleven representative skill strings mapped to expected role labels	Covered by separate pytest parametrized tests
Learning graph generation	External Gemini request and graph fields defined in repository code	No controlled test results supplied in current report
Practice grading	AI-supported score and feedback endpoint described in source	No human-rated quality benchmark supplied
Cross-module journey	Shared career-to-learning end-to-end workflow	Not validated by available test evidence

Verification note: the supplied command-line API test runner completed successfully in the preparation environment. It also emitted scikit-learn inconsistent-version warnings because saved artefacts were created under version 1.4.2 while the active environment used version 1.8.0. The warning must remain documented until the environment is pinned or the model artefacts are rebuilt.

APPENDIX III

CERTIFICATE





APPENDIX IV

INDIVIDUAL WORK CONTRIBUTION

PROJECT: DYNAMIC EVENT TICKETING MARKETPLACE WITH PREDICTIVE SCALPING MITIGATION ALGORITHMS

DEEPAK A (7376242AD136)

- Deepak built the TicketFlow back end in Java 17 and Spring Boot 3.5.6. He developed the REST APIs for login (JWT access and refresh tokens), event management, seat setup, reservations and booking confirmation. He also built the real-time updates using Server-Sent Events (SSE). These send live queue position, wait time and seat status to users .
- He implemented the anti-bot and fair-access features. The virtual queue uses Redis lists with normal and VIP lanes. The QueueDecisionService switches between direct access and a waiting queue based on system load. Admitted users get a temporary X-Queue-Token that is invalidated after booking. He also added the bot-detection checks and the ticket limits that block bulk and automated booking attempt .
- He made booking safe under heavy traffic. Redis distributed seat locks, sorted to avoid deadlocks, and Hibernate @Version optimistic locking prevent double booking. An Idempotency-Key header stops duplicate charges. Quartz jobs open and close sales on schedule. RabbitMQ sends confirmation emails with a dead-letter queue and retries, and Actuator with Prometheus monitors the system .

APPENDIX IV

INDIVIDUAL WORK CONTRIBUTION

PROJECT: DYNAMIC EVENT TICKETING MARKETPLACE WITH PREDICTIVE SCALPING MITIGATION ALGORITHMS

THIRUMURUGAN K (7376242AL210)

- Built the front end in React.js. He created the components for registration and login, event listing, the seat map, the queue waiting room, checkout and booking confirmation. The screens show live availability, queue position and booking status from the back-end APIs. The interface is simple and responsive.
- He implemented role-based access control. Buyers and administrators see different screens after login. Admin pages such as event creation and seat setup are shown only to ADMIN users. He also kept the JWT tokens on the client, sent them with each API request and protected routes that need login .
- Designed the SQL schema for the MySQL database. It covers users with roles, events, seats, reservations and bookings, with the fields the booking flow needs, such as seat status, reservation expiry and the version column for optimistic locking. He also tested the checkout journey from the front end to the back end, including failure cases such as an expired reservation or a failed payment s.

APPENDIX V

PLAGIARISM REPORT

